



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Sistem de Control FPGA pentru Mână Robotică cu Filtrare Kalman

LUCRARE DE LICENȚĂ

Absolvent: **Radu-Rareș CIOBANU**

Coordonator **Prof. dr. ing. Radu Gabriel DĂNESCU**
științific:

2026



FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN,
Prof. dr. ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT,
Prof. dr. ing. Rodica POTOLEA

Absolvent: **Radu-Rareș CIOBANU**

Sistem de Control FPGA pentru Mână Robotică cu Filtrare Kalman

1. **Enunțul temei:** *Proiectarea și implementarea unui sistem de control pe FPGA pentru o mână robotică cu 8 servomotoare, utilizând filtrare Kalman pentru stabilizarea mișcării și feedback de la un ADC pe 24 de biți.*
2. **Conținutul lucrării:** *Introducere, Obiectivele proiectului, Studiu bibliografic, Analiză și fundamentare teoretică, Proiectare de detaliu și implementare, Testare și validare, Manual de instalare și utilizare, Concluzii, Bibliografie, Anexe.*
3. **Locul documentării:** Universitatea Tehnică din Cluj-Napoca, Departamentul Calculatoare
4. **Consultanți:**
5. **Data emiterii temei:** 1 Octombrie 2025
6. **Data predării:** 10 Iulie 2026

Absolvent: _____

Coordonator științific: _____

**UNIVERSITATEA TEHNICĂ**
DIN CLUJ-NAPOCA**FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE**
DEPARTAMENTUL CALCULATOARE**Declarație pe propria răspundere privind
autenticitatea lucrării de licență**

Subsemnatul(a) _____, _____ legitimat(ă) cu

_____ seria _____ nr. _____
(conform copiei anexate prezentei declarații, copie sem-
nată și certificată “conform cu originalul”), autorul lucrării

elaborată în vederea susținerii examenului de finalizare a studiilor de licență la Facultatea de Automatică și Calculatoare, Programul de studiu _____ din cadrul Universității Tehnice din Cluj-Napoca, sesiunea _____ a anului universitar _____, declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele și pe baza informațiilor obținute din surse care au fost citate, în textul lucrării și în bibliografie.

Declar, că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administra-
tive, respectiv, *anularea examenului de licență*.

Data

Nume, Prenume

Semnătura

Cuprins

Capitolul 1	Introducere	1
1.1	Contextul lucrării	1
1.2	Motivația personală	1
1.3	Motivația alegerii platformei FPGA	2
1.4	Prezentarea generală a sistemului	2
Capitolul 2	Obiectivele proiectului	4
2.1	Preluarea și filtrarea datelor pentru mimarea fidelă a mișcării	4
2.2	Acționarea servomotoarelor pentru manipularea obiectelor	4
2.3	Funcționarea autonomă	5
2.4	Izolarea galvanică și siguranța	5
2.5	Arhitectură modulară și încadrarea în resursele disponibile	6
Capitolul 3	Studiu bibliografic	7
3.1	Proteze și mâini robotice: evoluție și clasificare	7
3.2	Anatomia mâinii și mâinile robotice biomimetice	8
3.3	Proiecte open-source de mâini robotice	10
3.3.1	InMoov	11
3.3.2	Modelul palmei	12
3.3.3	Antebrațul și cotul	13
3.3.4	Horizon	13
3.4	Filtrul Kalman	13
3.5	Servomotoare și controlul prin PWM	14
3.6	Fabricarea prin printare 3D	15
Capitolul 4	Analiză și fundamentare teoretică	16
4.1	Arhitectura sistemului	16
4.1.1	Lanțul de semnal	16
4.1.2	Ciclul de control de 20 ms	17
4.2	Protocolul de comunicație UART	17
4.2.1	Formatul cadrului	17
4.2.2	Mecanismul sumei de control XOR	18
4.2.3	Analiza temporală a transmisiei	18
4.2.4	Sincronizarea ceasului UART	19
4.3	Generarea semnalelor PWM	19
4.3.1	Specificația semnalului servo PWM	19
4.3.2	Cuantizarea la 100 MHz	19
4.3.3	Principiul de funcționare	20
4.4	Filtrul Kalman — fundamentare matematică	20
4.4.1	Formularea problemei	20
4.4.2	Modelul de stare	21
4.4.3	Pasul de predicție	22
4.4.4	Pasul de actualizare (corecție)	23
4.4.5	Ciclul predicție–actualizare: o perspectivă unitară	25
4.4.6	Alegerea parametrilor și semnificația lor fizică	25

4.5	Aritmetica Q16.16 în virgulă fixă	27
4.5.1	Formatul de reprezentare	27
4.5.2	Exemple de conversie	27
4.5.3	Operații aritmetice	28
4.5.4	Justificarea alegerii Q16.16 față de Q8.8	28
4.6	Interfața SPI și convertorul AD7124-8	29
4.6.1	Protocolul SPI — Modul 3	29
4.6.2	Convertorul AD7124-8	29
4.6.3	Modul DATA_STATUS	29
4.6.4	Conversia ADC → unghi	30
4.7	Bugetul de timp	30
4.7.1	Costul computațional per canal	30
4.7.2	Bugetul total pentru 8 canale	31
4.7.3	Marja de siguranță și extensibilitate	31
Capitolul 5 Proiectare de detaliu și implementare		33
5.1	Schema generală a sistemului	33
5.2	Modulul <code>system_tick_gen</code>	34
5.2.1	Interfața de porturi	34
5.2.2	Funcționare	34
5.3	Modulul <code>uart_rx</code> — Recepția UART	34
5.3.1	Calculul parametrilor de baud	34
5.3.2	Protecția contra metastabilității	35
5.3.3	Mașina de stări	35
5.4	Modulul <code>uart_tx</code> — Transmisia UART	36
5.4.1	Mașina de stări	36
5.5	Modulul <code>frame_rx</code> — Parserul de cadre	36
5.5.1	Mașina de stări	36
5.5.2	Mecanismul de timeout	37
5.5.3	Principiul verificării XOR	37
5.6	Modulul <code>frame_tx</code> — Transmisia cadrelor de răspuns	37
5.6.1	Mașina de stări	37
5.7	Modulul <code>mailbox</code> — Buffer dublu	38
5.7.1	Principiul ping-pong	38
5.8	Modulul <code>source_mux</code> — Multiplexor de surse	38
5.9	Modulul <code>demo_rom</code> — Secvențe demonstrative	38
5.9.1	Formatul cadru-cheie	39
5.9.2	Secvențele de mișcare	39
5.9.3	Logica de avansare	39
5.10	Modulul <code>spi_master</code> — Interfața SPI	39
5.10.1	Parametrii SPI mod 3	39
5.10.2	Mașina de stări	40
5.10.3	Protecția contra metastabilității pe MISO	40
5.11	Modulul <code>ad7124_ctrl</code> — Controlerul ADC	40
5.11.1	Faza de inițializare	40
5.11.2	Faza de citire ciclică	41
5.12	Modulul <code>kalman_engine</code> — Filtrul Kalman	42
5.12.1	Constantele de configurare	42

5.12.2	Memoria de stare (State RAM)	42
5.12.3	Funcția de multiplicare Q16.16	43
5.12.4	Mașina de stări a filtrului Kalman	43
5.12.5	Buget temporal	46
5.13	Modulul pwm_gen — Generatorul PWM	46
5.13.1	Generarea a 8 instanțe	46
5.14	Modulul basys3_top — Integrarea la nivel superior	47
5.14.1	Conectarea porturilor	47
5.14.2	Gestionarea pinilor neutilizați	47
5.14.3	Indicatoarele LED	48
5.14.4	Logica de declanșare a răspunsului	48
5.15	Proiectarea hardware — circuitele de interfață	48
5.15.1	Izolarea galvanică a semnalelor PWM — ADUM1400	48
5.15.2	Alimentarea servomotoarelor — DC2468A	49
5.15.3	Alimentarea circuitului ADC — ADP165	49
5.15.4	Izolarea SPI — MAX14850	49
5.15.5	Conectarea servomotoarelor MG996R	49
Capitolul 6	Testare și validare	50
6.1	Metodologia de testare	50
6.2	Testarea modulului PWM	50
6.2.1	Metodă	50
6.2.2	Cazuri de test	51
6.2.3	Analiza rezultatelor	51
6.3	Testarea comunicației UART	51
6.3.1	Testul de loopback	51
6.3.2	Procedura de test	51
6.3.3	Rezultate	52
6.4	Testarea protocolului de cadre	52
6.4.1	Verificarea recepției cadrelor	52
6.4.2	Verificarea transmisiei cadrelor de răspuns	53
6.5	Testarea SPI și ADC	53
6.5.1	Verificarea interfeței SPI	53
6.5.2	Verificarea comunicării cu AD7124-8	53
6.5.3	Verificarea scanării canalelor	53
6.6	Testarea filtrului Kalman	53
6.6.1	Testul de convergență	53
6.6.2	Testul de răspuns la treaptă	54
6.6.3	Testul de rejectie a zgomotului	54
6.7	Testarea modului demo	54
6.7.1	Procedură	54
6.7.2	Rezultate	54
6.8	Testare de integrare	55
6.8.1	Lanțul complet de semnal	55
6.8.2	Măsurarea latenței end-to-end	55
6.8.3	Verificarea indicatoarelor LED	55
6.9	Utilizarea resurselor FPGA	56
6.9.1	Analiza utilizării	56

6.9.2	Distribuția resurselor per modul	56
Capitolul 7	Manual de instalare și utilizare	58
7.1	Cerințe hardware	58
7.2	Cerințe software	58
7.3	Instalarea mediului de dezvoltare	59
7.3.1	Crearea proiectului RTL	59
7.3.2	Rezolvarea problemelor comune	59
7.4	Programarea plăcii	60
7.5	Conectarea hardware	60
7.5.1	Conectarea servomotoarelor prin izolatori	60
7.5.2	Alimentarea	60
7.6	Utilizarea sistemului	61
7.6.1	Modul demonstrativ	61
7.6.2	Modul UART (control de la PC)	61
7.6.3	Resetarea sistemului	61
7.6.4	Interpretarea indicatoarelor LED	62
Capitolul 8	Concluzii	63
8.1	Contribuții	63
8.2	Analiză critică	63
8.2.1	Puncte forte	63
8.2.2	Limitări	64
8.3	Dezvoltări ulterioare	64
Bibliografie		66
Anexa A	Codul sursă VHDL	68

Capitolul 1. Introducere

1.1. Contextul lucrării

Protezele robotice reprezintă unul dintre cele mai active domenii de cercetare la intersecția dintre ingineria calculatoarelor, biomedicală și robotică. Capacitatea de a reproduce mișcările naturale ale mâinii umane are implicații profunde atât în domeniul medical, restabilirea funcționalității membrilor superioare pentru persoanele cu dizabilități, cât și în cel industrial, unde mâinile robotice sunt utilizate în manipularea obiectelor, asamblare de precizie și interacțiune om-robot.

Un aspect esențial al acestor sisteme este *controlul în timp real*. Mâna umană posedă 27 DOF (*Degrees of Freedom*, numărul de mișcări independente posibile) și poate efectua mișcări cu o latență de ordinul milisecundelor. Pentru a reproduce un comportament natural, un sistem robotic trebuie să proceseze datele senzoriale, să determine pozițiile dorite ale articulațiilor și să genereze semnalele de control în cadrul unor constrângeri temporale stricte. Orice latență sau fluctuație în semnalele de comandă se traduce în mișcări sacadate, nepotrivite cu gestul intenționat.

În paralel, domeniul recunoașterii gesturilor a cunoscut progrese semnificative datorită dezvoltării rețelelor neuronale și a bibliotecilor de viziune computerizată. Google MediaPipe [1] oferă un model de estimare a pozițiilor articulațiilor mâinii în timp real, capabil să funcționeze pe hardware convențional. Acest lucru deschide posibilitatea construirii unor sisteme de teleoperare accesibile, în care un operator uman controlează o mână robotică prin simpla filmare a propriei mâini cu o cameră web.

Provocarea principală constă în integrarea celor două domenii: pe de o parte, procesarea video și extragerea coordonatelor articulațiilor (realizată pe un calculator gazdă) și, pe de altă parte, controlul determinist al servomotoarelor (realizat pe o platformă hardware dedicată). Această lucrare abordează tocmai această integrare, propunând o soluție bazată pe FPGA (*Field-Programmable Gate Array*, matrice de porți programabile) pentru componenta de control.

1.2. Motivația personală

Pasiunea pentru construirea de lucruri concrete și plăcerea de a vedea un proiect prinzând viață m-au îndreptat către o temă de licență cu un puternic caracter aplicativ. Mi-am dorit să realizez ceva util și, în același timp, spectaculos, iar domeniul teleoperării m-a fascinat dintotdeauna prin ideea de a controla de la distanță un sistem mecanic care reproduce gesturile umane. Din această dorință s-a născut tema lucrării de față: un braț robotic care imită în timp real mișcările mâinii umane, preluate cu ajutorul unei camere web. Componenta de viziune computerizată este dezvoltată de colegul meu Lion Moroz, în timp ce contribuția mea acoperă preluarea datelor prin UART (*Universal Asynchronous Receiver-Transmitter*) și prelucrarea lor pe placa Basys3, unde sunt implementate filtrul Kalman și celelalte module de control descrise în această lucrare. Pentru realizarea sistemului am utilizat, de asemenea, o gamă largă de componente Analog Devices, după cum se va observa pe parcurs.

Neavând experiență în modelarea 3D, am pornit de la modele open-source exis-

tente. Primul care mi-a atras atenția a fost brațul prezentat de Horizon într-un material video [2]. I-am scris autorului pentru a-i solicita fișierele, iar acesta a răspuns și le-a publicat pe platforma Thingiverse. Deși ulterior am ales un alt model, mai potrivit cerințelor proiectului, această interacțiune cu comunitatea open-source mi-a oferit un impuls și o motivație suplimentară pentru a finaliza un proiect de calitate.

1.3. Motivația alegerii platformei FPGA

Alegerea unei platforme FPGA pentru controlul servomotoarelor, în locul unui microcontroler tradițional, este motivată de mai multe avantaje fundamentale.

Determinism temporal. Un microcontroler execută instrucțiuni secvențial, iar întreruperile, schimbările de context și accesul la memorie introduc variabilitate în timpul de execuție (*jitter*). Într-un FPGA, logica de control este implementată direct în hardware, iar semnalele PWM (*Pulse Width Modulation*, modulație în lățimea pulsului) sunt generate de circuite dedicate care funcționează sincron cu ceasul de sistem. Rezultatul este un jitter de ordinul nanosecundelor, comparativ cu microsecunde sau chiar zecile de microsecunde ale unui microcontroler. Pentru servomotoare de tip MG996R, care interpretează lățimea pulsului PWM ca poziție angulară, această precizie temporală se traduce direct în stabilitatea poziționării.

Paralelism real. Un FPGA permite generarea simultană a tuturor celor 8 canale PWM, fără multiplexare în timp. Fiecare canal PWM dispune de propriul comparator hardware, iar actualizarea poziției unui servo nu afectează în niciun fel celelalte canale. În plus, modulul de filtrare Kalman, interfața UART și controlerul SPI (*Serial Peripheral Interface*) pentru ADC-ul (*Analog-to-Digital Converter*, convertor analog-digital) AD7124-8 funcționează toate în paralel, fără a concura pentru același procesor.

Filtrare Kalman în timp real. Filtrul Kalman este un estimator optim pentru sisteme liniare cu zgomot gaussian. În contextul acestei lucrări, acesta modelează fiecare servo ca un sistem cu două stări (poziție și viteză) și estimează traiectoria reală pe baza măsurătorilor zgomotoase primite de la MediaPipe. Implementarea în FPGA, utilizând aritmetică în virgulă fixă Q16.16, permite execuția completă a algoritmului într-un număr fix de cicluri de ceas, garantând că filtrarea se finalizează înainte de următorul ciclu de servo (20 ms).

Scalabilitate. Arhitectura modulară a proiectului permite adăugarea de noi canale PWM sau senzori fără modificarea logicii existente, în limita resurselor FPGA disponibile.

1.4. Prezentarea generală a sistemului

Sistemul propus în această lucrare realizează controlul unei mâini robotice printate 3D, cu 8 servomotoare MG996R, pe baza gesturilor capturate în timp real de la un operator uman. Arhitectura de ansamblu este structurată pe două niveluri:

1. **Nivelul de achiziție:** un calculator gazdă rulează biblioteca MediaPipe pentru detecția articulațiilor mâinii din fluxul video. Unghiurile calculate pentru fiecare articulație sunt transmise prin UART la 115200 baud, sub forma unor cadre de 10 octeți.
2. **Nivelul de control:** placa FPGA Basys3 (Xilinx Artix-7 XC7A35T, 100 MHz) recepționează cadrele UART, aplică un filtru Kalman cu 2 stări pentru fiecare canal, generează 8 semnale PWM independente și transmite aceste semnale prin izolatori galvanici ADUM1400 către servomotoare. Un ADC AD7124-8 pe 24 de biți, interfațat prin SPI printr-un izolator MAX14850, permite citirea poziției reale

a servomotoarelor pentru verificare.

Lanțul complet de semnal este:

Camera → MediaPipe → UART → FPGA → ADUM1400 → Servo

Cele 8 servomotoare sunt mapate pe articulațiile mâinii robotice după cum urmează: degetul mic, degetul inelar, degetul mijlociu, degetul arătător, mișcarea palmară a degetului mare, tensiunea degetului mare, rotația încheieturii și extensia cotului. Această configurație permite reproducerea majorității gesturilor naturale ale mâinii.

Un mod demonstrativ (*demo mode*), activabil prin intermediul comutatoarelor de pe placa Basys3, permite funcționarea autonomă a sistemului prin parcurgerea unei secvențe predefinite de mișcări stocate într-un modul ROM, fără a necesita conexiunea cu calculatorul gazdă.

Capitolul 2. Obiectivele proiectului

Proiectul de față își propune realizarea unui sistem de control pe FPGA pentru o mână robotică acționată de 8 servomotoare. Sistemul primește, printr-o interfață serială, unghiurile articulațiilor unei mâini umane urmărite de un sistem extern de viziune și le transformă în comenzi de poziție pentru servomotoare. Pentru ca mișcarea reprodușă să fie naturală și lipsită de tremur, datele primite sunt prelucrate de un filtru Kalman înainte de a fi trimise spre actuatori. Pe lângă modul de operare comandat extern, sistemul poate funcționa și autonom, parcurgând secvențe de mișcare predefinite, iar separarea dintre logica de control și partea de putere este asigurată prin izolare galvanică.

2.1. Preluarea și filtrarea datelor pentru mimarea fidelă a mișcării

Obiectivul principal al proiectului este preluarea unghiurilor articulațiilor printr-o interfață serială și prelucrarea acestora astfel încât mâna robotică să reproducă cât mai fidel mișcarea mâinii umane. Datele de intrare descriu, pentru fiecare articulație, unghiul dorit, iar sistemul trebuie să convertească aceste valori în comenzi de poziție stabile, transmise în mod continuu către servomotoare. Calitatea acestei reproduceri reprezintă criteriul central după care este evaluat întregul sistem.

Datele primite de la sistemul de viziune nu sunt însă perfecte, deoarece conțin zgomot și variații de la un cadru la altul, cauzate de erorile inerente ale procesului de detecție. Dacă aceste valori ar fi trimise direct la servomotoare, mișcarea rezultată ar fi sacadată, cu tremurături vizibile și nepotrivite cu gestul real. Pentru a corecta acest neajuns, sistemul aplică un filtru Kalman, care estimează poziția reală a fiecărei articulații pe baza valorilor zgomotoase primite și produce o traiectorie netedă și continuă. Filtrarea Kalman constituie astfel elementul central al lucrării, fiind componenta care transformă un flux brut de date într-o mișcare fluidă, apropiată de cea naturală.

2.2. Acționarea servomotoarelor pentru manipularea obiectelor

Un obiectiv secundar al proiectului este acționarea precisă a celor 8 servomotoare, astfel încât mâna robotică să poată executa gesturi utile, inclusiv apucarea și manipularea unor obiecte. Fiecare servomotor este controlat printr-un semnal de tip PWM, a cărui durată (*pulse width*) determină poziția unghiulară a articulației corespunzătoare. Pentru a obține o mișcare coordonată a degetelor, toate cele 8 semnale trebuie generate simultan și actualizate sincron, fără întârzieri între canale. Tocmai aici intervine avantajul platformei FPGA. Spre deosebire de un microcontroler, care ar genera semnalele secvențial, FPGA-ul permite implementarea a 8 module PWM independente, ce funcționează în paralel și produc cele 8 semnale în mod perfect simultan.

Acuratețea și stabilitatea acestor semnale sunt esențiale deoarece o variație nedorită a duratei pulsului se traduce printr-o tremură a articulației, iar lipsa sincronizării între canale ar face ca degetele să se miște la momente ușor diferite, compromițând prinderea fermă a unui obiect. Întrucât în FPGA fiecare semnal este generat de un circuit dedicat, sincronizat cu ceasul de sistem, acestea sunt precise și independente, îndeplinind cerințele acestui obiectiv.

Pentru manipularea efectivă a obiectelor, nu este suficient ca fiecare deget să ajungă într-o poziție dorită, ci este nevoie ca toate degetele să își coordoneze mișcarea într-un gest unitar. Apucarea unui obiect presupune ca degetele să se închidă progresiv și împreună, până când obiectul este prins ferm. Dacă un singur canal ar fi întârziat sau instabil, prinderea ar deveni nesigură, iar obiectul ar putea aluneca. Din acest motiv, generarea simultană și sincronă a tuturor celor 8 semnale nu este doar o cerință tehnică, ci condiția fundamentală pentru ca mâna robotică să poată îndeplini sarcini reale de manipulare.

2.3. Funcționarea autonomă

Un alt obiectiv secundar este capacitatea sistemului de a funcționa autonom, fără a depinde de conexiunea cu un calculator extern. În acest mod, mâna robotică parcurge o secvență de mișcări predefinite, stocate intern, permițând demonstrarea funcționalității sistemului în lipsa sursei de date externe.

Acest mod de operare este util atât pentru testarea independentă a părții de control și actuare, cât și pentru prezentarea sistemului în condiții în care echipamentul de captură nu este disponibil. Comutarea între modul comandat extern și cel autonom trebuie să fie simplă și să nu necesite reconfigurarea sistemului.

Secvențele predefinite urmăresc demonstrarea principalelor capabilități ale mâinii robotice, printre care deschiderea și închiderea completă a degetelor, mișcările individuale ale fiecărui deget, precum și gesturi uzuale. În acest fel, modul autonom oferă o imagine clară asupra întregului interval de mișcare al sistemului și asupra funcționării corecte a fiecărui servomotor în parte, fără a depinde de o sursă externă de date.

Din punct de vedere practic, acest obiectiv prezintă și un avantaj important pe parcursul dezvoltării. Întrucât partea de viziune și partea de control sunt realizate separat, modul autonom permite verificarea completă a sistemului de control înainte de integrarea cu sistemul de captură, eliminând dependența de disponibilitatea acestuia din urmă. Astfel, eventualele probleme de actuare pot fi identificate și corectate independent, ceea ce simplifică procesul de testare și validare.

2.4. Izolarea galvanică și siguranța

Deoarece sistemul cuprinde două domenii electrice distincte, partea de logică de control la tensiune redusă și partea de putere care alimentează servomotoarele, un obiectiv secundar important este asigurarea unei separări galvanice între acestea. Această separare protejează componenta digitală de eventualele perturbații, vârfuri de curent sau zgomot generate de servomotoare în timpul funcționării.

Necesitatea acestei separări devine evidentă dacă se ține cont de comportamentul electric al servomotoarelor. La pornire și în timpul mișcării, acestea pot genera vârfuri de curent și perturbații pe linia de alimentare, fenomene care, în absența unei separări, s-ar putea propaga către placa de control și ar putea afecta funcționarea logicii digitale sau chiar deteriora componentele acesteia.

Izolarea galvanică asigură atât siguranța în funcționare, cât și integritatea semnalelor transmise între cele două domenii. În acest scop, semnalele de comandă către servomotoare și cele de comunicație sunt transferate prin componente dedicate de izolare, care permit trecerea informației fără a exista o legătură electrică directă între cele două părți. Îndeplinirea acestui obiectiv contribuie la fiabilitatea generală a sistemului și la protejarea plăcii de control împotriva defectelor care ar putea proveni din partea de putere.

2.5. Arhitectură modulară și încadrarea în resursele disponibile

Un ultim obiectiv secundar vizează organizarea sistemului într-o arhitectură modulară, în care fiecare funcție (comunicație, filtrare, generare a semnalelor, control) este implementată ca un bloc independent. O astfel de structură ușurează dezvoltarea, testarea și eventuala extindere ulterioară a sistemului, fiecare modul putând fi verificat separat.

În plus, întreaga implementare trebuie să se încadreze în resursele limitate ale plăcii FPGA utilizate și să respecte constrângerile temporale impuse de funcționarea în timp real. Respectarea acestor limite reprezintă o cerință de proiectare permanentă, care influențează deciziile luate pe parcursul realizării fiecărui modul.

Capitolul 3. Studiu bibliografic

Acest capitol prezintă stadiul actual al domeniului în care se încadrează lucrarea de față. Sunt trecute în revistă evoluția protezelor și a mâinilor robotice, principiile anatomice și mecanice care stau la baza reproducerii mișcărilor mâinii, precum și tehnologiile pe care se sprijină proiectul, de la filtrarea Kalman până la actuarea cu servomotoare și fabricarea prin printare 3D.

3.1. Proteze și mâini robotice: evoluție și clasificare

Preocuparea omului de a înlocui un membru pierdut este surprinzător de veche. Primele forme de proteze sunt atestate încă din Antichitate, fiind descoperite, de exemplu, proteze de deget realizate din lemn și piele atașate unor mumii egiptene, dovadă a faptului că dorința de a restabili, fie și parțial, funcționalitatea sau aspectul unui membru a existat dintotdeauna [3]. De la aceste dispozitive rudimentare și până la sistemele complexe de astăzi, controlate electronic, protezele au parcurs un drum lung, fiecare etapă fiind marcată de tehnologiile disponibile în epoca respectivă.



Figura 3.1: Proteză de deget din lemn și piele, descoperită la o mumie egipteană, una dintre cele mai vechi proteze cunoscute [4].

Cele mai simple proteze, numite proteze pasive, nu au părți mobile și urmăresc în principal refacerea aspectului estetic, oferind o funcționalitate de bază. Un pas mai departe îl reprezintă protezele mecanice, acționate chiar de corpul utilizatorului prin intermediul unui sistem de hamuri legat de mișcările umărului sau ale cotului, iar această mișcare este transmisă mai departe printr-un cablu către un cârlig sau o mână mecanică. Deși sunt dispozitive relativ simple, ele rămân și astăzi printre cele mai răspândite proteze, tocmai datorită robusteții și a costului redus [3].



Figura 3.2: Proteze mecanice cu cârlig, acționate de corp prin hamuri și cabluri [5].

Odată cu dezvoltarea electronicii a apărut o categorie mult mai performantă, aceea a protezelor mioelectrice. Acestea funcționează măsurând semnalele electrice produse de contracția mușchilor rămași în zona membrului afectat, semnale preluate prin electrozi, amplificate și transmise unui microcontroler care comandă apoi actuatorii protezei. În acest fel, utilizatorul capătă un control mult mai fin decât în cazul protezelor mecanice. La granița actuală a cercetării se află interfețele neuronale directe, în care semnalele sunt preluate chiar de la nivelul creierului prin electrozi implantați chirurgical. Deși o astfel de tehnologie este încă la început, ea a demonstrat deja că o persoană poate comanda un braț robotic doar prin gândire [3].



Figura 3.3: Proteză modernă, acționată electronic, cu un grad ridicat de control asupra degetelor [6].

Sistemul realizat în această lucrare nu este o proteză mioelectrică în sensul clasic, ci o mână robotică acționată în funcție de mișcările unei mâini umane urmărite vizual. Cu toate acestea, el se bazează pe aceleași principii mecanice și de control întâlnite la protezele moderne, iar înțelegerea acestora oferă contextul necesar pentru restul lucrării.

3.2. Anatomia mâinii și mâinile robotice biomimetice

Pentru a reproduce mișcările unei mâini este util să pornim de la complexitatea modelului pe care încercăm să îl imităm. Mâna umană este alcătuită din cel puțin 27 de oase, peste 30 de mușchi și mai mult de o sută de ligamente, nervi și artere, ceea ce o face un mecanism extrem de complex [3]. Tocmai de aceea, niciun sistem robotic actual nu

reușește să egaleze dexteritatea și fluiditatea unei mâini reale, iar orice proiect din acest domeniu pornește inevitabil de la o variantă simplificată a ei.

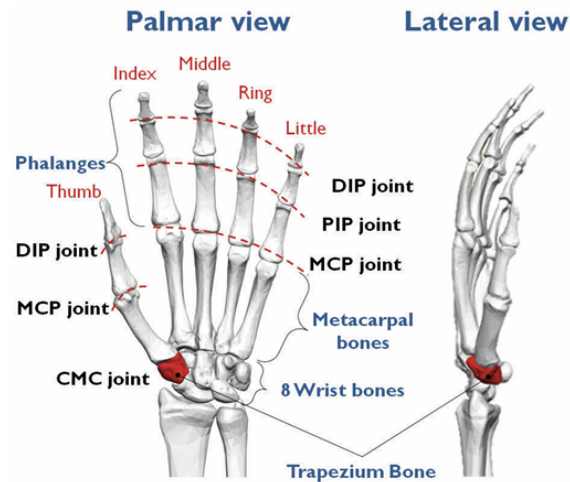


Figura 3.4: Structura osoasă a mâinii umane și articulațiile degetelor (DIP, PIP, MCP și CMC) [7].

Un concept esențial pentru a descrie mișcarea unei astfel de structuri este cel de grad de libertate (DOF), prin care se înțelege o modalitate independentă în care un element se poate mișca. Un deget uman, de pildă, dispune de patru grade de libertate. Trei dintre ele provin din rotația articulațiilor sale și controlează împreună îndoirea și întinderea degetului, iar al patrulea permite mișcarea laterală, de depărtare și apropiere față de celelalte degete [3]. Degetul mare beneficiază de un grad suplimentar, ajungând astfel la cinci. Aceste mișcări sunt produse, în corpul uman, prin contracția mușchilor și a tendoanelor, care trag de oase și le deplasează.

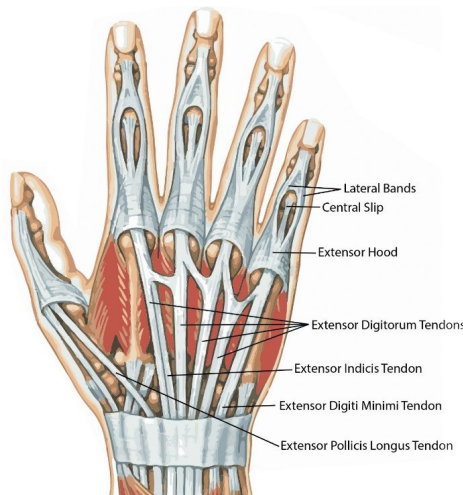


Figura 3.5: Rețeaua de tendoane extensoare de la nivelul mâinii, responsabilă de întinderea degetelor [8].

Mai exact, mișcarea degetelor este controlată de două grupe de tendoane care acționează în sens opus. Tendoanele extensoare, situate pe partea dorsală a mâinii, întind

degetul, în timp ce tendoanele flexoare, de pe partea palmară, îl îndoaie. Tendoanele flexoare sunt ghidate de-a lungul degetului prin niște teci numite scripeti (*pulleys*), care le mențin aproape de os și asigură o îndoire controlată [7]. Această pereche flexor-extensor este exact principiul pe care îl reproduce un sistem robotic acționat prin tendoane.

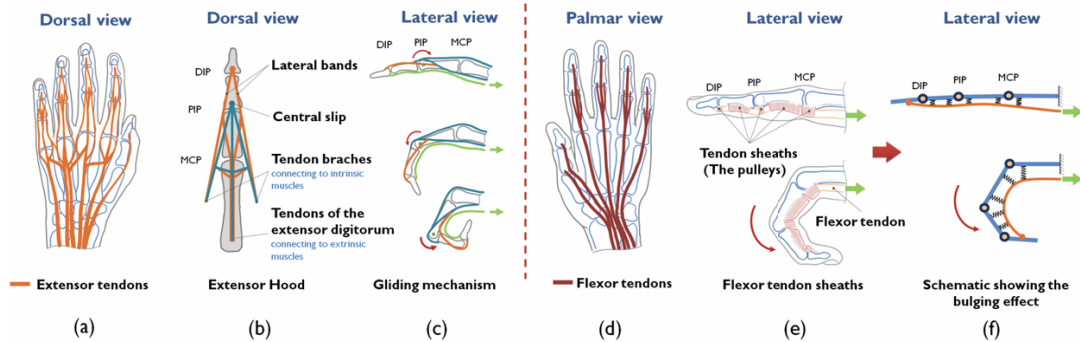


Figura 3.6: Mecanismul tendoanelor la nivelul degetului: tendoanele extensoare (a–c) întind degetul, iar cele flexoare (d–f), ghidate prin scripeti, îl îndoaie [7].

Acest mecanism natural a inspirat o întreagă categorie de mâini robotice, numite biomimetice, care imită structura mâinii umane folosind tendoane artificiale ce acționează degetele prin tragere, reproducând astfel îndoirea și întinderea lor naturală [7]. Atractivitatea acestei abordări vine din faptul că oferă o metodă relativ simplă și ieftină de a controla degetele, fără a recurge la mecanisme complicate.

Spre deosebire de aceasta, majoritatea protezelor comerciale folosesc un sistem de articulații cuplate mecanic și acționate de motoare electrice [3]. Particularitatea lor este că toate articulațiile unui deget sunt comandate de un singur motor, ceea ce înseamnă că întregul deget se poate mișca într-un singur mod, deschizându-se și închizându-se la fel de fiecare dată. Practic, deși au tot atâtea articulații ca un deget uman, aceste degete dispun de mult mai puține grade de libertate, de obicei unul sau două față de cele patru ale degetului real. Această limitare explică de ce protezele comerciale pot îndeplini doar sarcini relativ simple și nu pot reproduce varietatea de gesturi pe care le permite o mână adevărată.

Există însă și lucrări de cercetare care urmăresc apropierea cât mai mare de modelul biologic. Una dintre ele propune o mână robotică antropomorfă cu un grad ridicat de biomimetism, care reproduce fidel structura și mișcările mâinii umane [7]. Un astfel de proiect a constituit o sursă de inspirație pentru conceptele preluate în lucrarea de față, în special pentru modul în care mișcările sunt distribuite pe articulații. Reproducerea integrală a unei asemenea structuri presupune însă costuri ridicate și o complexitate mecanică greu de atins în cadrul unui proiect de licență, motiv pentru care soluția aleasă aici pornește de la o variantă mai simplă, construită din componente accesibile.

3.3. Proiecte open-source de mâini robotice

Dincolo de cercetarea academică și de produsele comerciale, în ultimii ani s-a dezvoltat o comunitate amplă de proiecte open-source dedicate mâinilor și brațelor robotice. Acestea pun la dispoziție gratuit modele tridimensionale care pot fi printate 3D și asamblate de oricine, fiind însoțite de obicei de o documentație extinsă și de o comunitate

activă de pasionați. Pentru proiectul de față, aceste resurse au reprezentat punctul de plecare, fiecare contribuind cu o parte din ansamblul final. În continuare sunt prezentate proiectele studiate și ceea ce a fost preluat din fiecare.

3.3.1. InMoov

Cel mai cunoscut proiect open-source din acest domeniu este InMoov, primul robot umanoid de dimensiune reală conceput pentru a fi în întregime printat 3D [9]. Datorită documentației extinse și a comunității active care s-a format în jurul său, InMoov a devenit un punct de referință pentru proiectele de mâini robotice accesibile. Mâna sa utilizează aproximativ șase servomotoare și acționează degetele prin tendoane, fiecare deget fiind tras de un cablu.

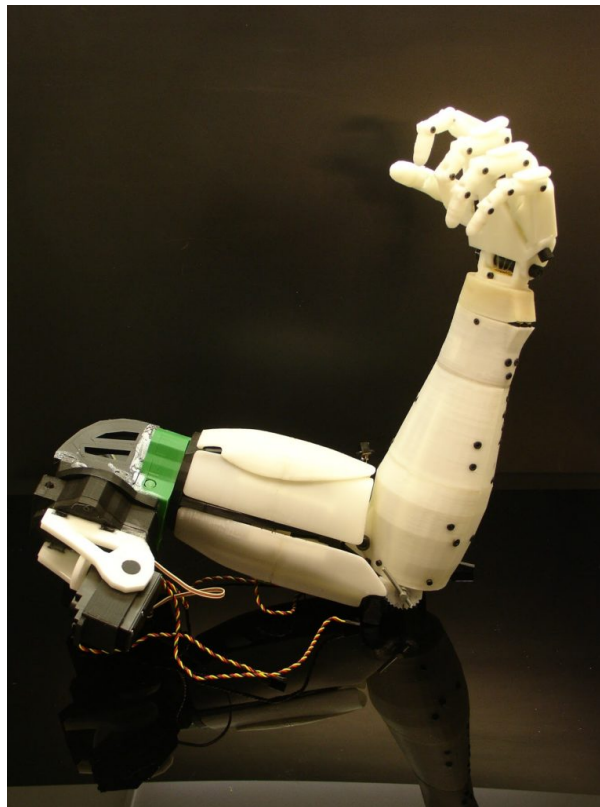


Figura 3.7: Brațul robotic InMoov, printat 3D [9].

Din acest proiect am preluat soluția pentru rotația încheieturii, care mi s-a părut cea mai bine gândită din punctul de vedere al ghidării firelor. Sistemul de scripeți care rotesc mâna s-a dovedit deosebit de util și ușor de integrat în restul ansamblului, motiv pentru care l-am păstrat aproape neschimbat.

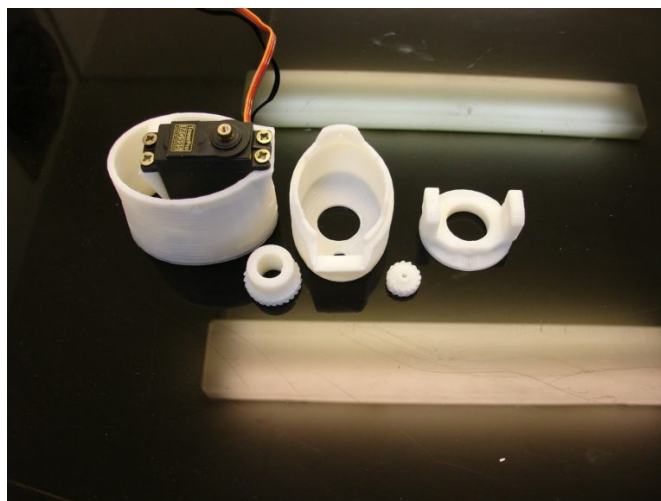


Figura 3.8: Mecanismul de rotație a încheieturii din proiectul InMoov, format dintr-un servomotor și componente printate 3D [9].

3.3.2. Modelul palmei

Palma și degetele propriu-zise au fost preluate de la un model dezvoltat de Ryonicle, disponibil public pe platforma Thingiverse [10]. Acest model mi-a atras atenția prin forma sa îngrijită și prin modul de acționare a degetelor: fiecare deget este tras de un servomotor prin intermediul unui tendon, iar revenirea în poziția inițială se face cu ajutorul unor elastice care joacă rolul tendoanelor extensoare. Această soluție reproduce, într-o formă simplificată, perechea flexor-extensor întâlnită la mâna umană.

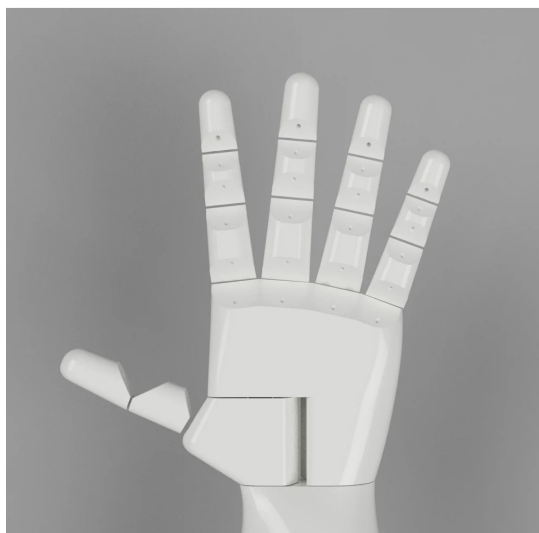


Figura 3.9: Modelul de mână dezvoltat de Ryonicle, de la care au fost preluate palma și degetele [10].

Modelul original folosea însă un alt tip de servomotoare decât cele alese în proiectul de față, ceea ce a impus câteva adaptări la nivelul locașurilor și al modului de prindere.

3.3.3. Antebrațul și cotul

Pentru zona de la încheietură în jos, mai exact întreg antebrațul și articulația cotului, împreună cu sistemul de scripeți aferent, am pornit de la modelul propus de Mahdi Hussein în teza sa dedicată unui braț protetic printat 3D [3]. Această parte găzduiește servomotoarele care acționează degetele și asigură traseul tendoanelor către mână, integrându-se bine cu restul ansamblului.

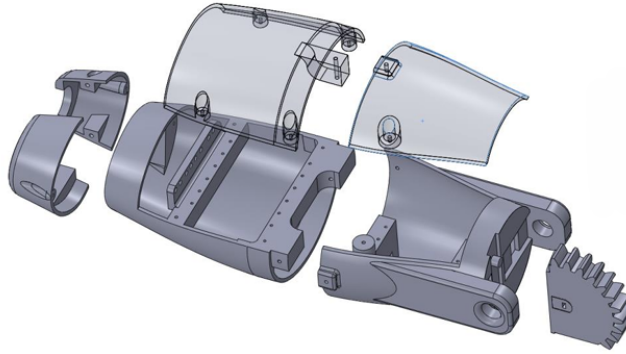


Figura 3.10: Modelul antebrațului propus de Hussein, preluat pentru zona de la încheietură în jos [3].

3.3.4. Horizon

Un alt proiect avut în vedere a fost cel prezentat de Horizon [2]. Deși am purtat o corespondență cu autorul pentru a obține fișierele necesare, modelul său de mână, de tip osos, nu se potrivea cu obiectivul meu de a realiza un braț aproape complet, fără umăr. Combinarea lui cu celelalte componente s-ar fi făcut cu dificultate, iar stabilitatea mișcărilor ar fi avut de suferit. În plus, cadrul propus de el era de dimensiuni mari și greu de adaptat la restul ansamblului, motiv pentru care, în final, nu a fost folosit.

Aceste proiecte au constituit baza de la care a pornit soluția proprie, un ansamblu hibrid ce preia de la fiecare sursă partea cea mai potrivită, iar modul concret în care au fost combinate este detaliat în capitolul de proiectare și implementare.

3.4. Filtrul Kalman

Componenta centrală a lucrării de față, cea care prelucrează datele de mișcare înainte de a fi trimise spre servomotoare, este un filtru Kalman. Acesta este un algoritm de estimare propus de Rudolf Kalman în anul 1960 [11] și a devenit, de atunci, unul dintre cele mai utilizate instrumente pentru estimarea stării unui sistem dinamic afectat de zgomot. El este folosit pe scară largă în domenii precum navigația, robotica sau procesarea semnalelor, oriunde este nevoie de o estimare cât mai exactă a unei mărimi pornind de la măsurători imperfecte.

Ideea de bază a filtrului este aceea de a combina două surse de informație, fiecare imperfectă în felul ei. Pe de o parte există o predicție, obținută pe baza unui model matematic al sistemului, care spune cum ar trebui să evolueze starea în timp. Pe de altă parte există o măsurătoare reală, dar afectată de zgomot. Filtrul cântărește aceste două surse în funcție de cât de mult se poate avea încredere în fiecare și produce o estimare finală mai bună decât oricare dintre ele luată separat. Rezultatul este o valoare netezită,

care urmărește fidel evoluția reală a sistemului, eliminând în același timp variațiile bruște cauzate de zgomot.

Din punct de vedere matematic, sistemul este descris printr-un vector de stare $\mathbf{x}$, care reține mărimile estimate, și printr-o matrice de covarianță $\mathbf{P}$, care exprimă incertitudinea asociată acestor estimări. Algoritmul funcționează ciclic, alternând două etape. În etapa de predicție, starea și incertitudinea sunt proiectate înainte în timp pe baza modelului sistemului, descris de matricea de tranziție $\mathbf{F}$ și de zgomotul de proces $\mathbf{Q}$ [12]:

$$\hat{\mathbf{x}}_k^- = \mathbf{F} \hat{\mathbf{x}}_{k-1} \quad (3.1)$$

$$\mathbf{P}_k^- = \mathbf{F} \mathbf{P}_{k-1} \mathbf{F}^\top + \mathbf{Q} \quad (3.2)$$

În etapa de actualizare, predicția este corectată cu ajutorul măsurătorii $\mathbf{z}_k$. Se calculează mai întâi câștigul Kalman $\mathbf{K}_k$, care stabilește cât de mult se ține cont de măsurătoare față de predicție, după care starea și covarianța sunt corectate [12]:

$$\mathbf{K}_k = \mathbf{P}_k^- \mathbf{H}^\top (\mathbf{H} \mathbf{P}_k^- \mathbf{H}^\top + \mathbf{R})^{-1} \quad (3.3)$$

$$\hat{\mathbf{x}}_k = \hat{\mathbf{x}}_k^- + \mathbf{K}_k (\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^-) \quad (3.4)$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k \mathbf{H}) \mathbf{P}_k^- \quad (3.5)$$

Aici $\mathbf{H}$ este matricea de observație, care leagă starea de mărimea măsurată, $\mathbf{R}$ reprezintă zgomotul de măsurare, iar $\mathbf{I}$ este matricea identitate. Diferența $\mathbf{z}_k - \mathbf{H} \hat{\mathbf{x}}_k^-$, numită inovație, arată cât de mult se abate măsurătoarea de la predicție, iar câștigul Kalman decide proporția în care această abatere corectează estimarea.

Pentru documentarea pe acest subiect, înțelegerea intuitivă a algoritmului a fost facilitată de prezentarea accesibilă realizată de Alan Zucconi [13], în timp ce fundamentele teoretice riguroase au fost aprofundate pe baza lucrărilor clasice din domeniu [12]. Modul concret în care filtrul a fost adaptat și implementat în cadrul acestui proiect este detaliat în capitolul de analiză și fundamentare teoretică.

3.5. Servomotoare și controlul prin PWM

Mișcarea efectivă a degetelor și a articulațiilor brațului robotic este realizată de servomotoare. Un servomotor este un actuator format dintr-un motor electric, un sistem de angrenaje și un circuit intern de control, care îi permite să se deplaseze într-o poziție unghiulară dorită și să o mențină. Spre deosebire de un motor obișnuit, care se rotește continuu, servomotorul primește o comandă de poziție și se deplasează exact în punctul indicat, ceea ce îl face potrivit pentru controlul articulațiilor unei mâini robotice.

Comanda de poziție se transmite printr-un semnal de tip PWM. Acesta este un semnal periodic la care nu amplitudinea, ci durata pulsului activ codifică informația. În cazul servomotoarelor uzuale, semnalul are o perioadă de 20 de milisecunde, iar durata pulsului variază în general între 1 și 2 milisecunde. Un puls scurt corespunde unei poziții la o extremă a cursei, un puls lung corespunde celeilalte extreme, iar valorile intermediare determină proporțional unghiul dorit. Circuitul intern al servomotorului compară permanent poziția curentă cu cea comandată și acționează motorul până când cele două coincid.

Pentru acest proiect a fost ales servomotorul MG996R, un model larg răspândit, cu angrenaje metalice și un cuplu de aproximativ 11 kg·cm [14]. Alegerea s-a bazat în principal pe cuplul ridicat, necesar pentru a trage de tendoanele care acționează degetele

și pentru a învinge rezistența elementelor elastice de revenire. Caracteristicile detaliate ale acestui servomotor, precum și modul în care sunt generate semnalele PWM în cadrul sistemului, sunt prezentate în capitolele următoare.

3.6. Fabricarea prin printare 3D

Componentele mecanice ale brațului robotic sunt realizate prin printare 3D, o tehnologie de fabricație aditivă în care obiectul este construit strat cu strat, pornind de la un model tridimensional. Cea mai răspândită variantă, folosită și în acest proiect, este depunerea de material topit (FDM, *Fused Deposition Modeling*), în care un filament de material plastic este încălzit și depus succesiv de o duză care se deplasează după contururile fiecărui strat. Această metodă a devenit accesibilă și ieftină, permițând realizarea unor piese complexe într-un timp relativ scurt, motiv pentru care este preferată în majoritatea proiectelor de mâini robotice open-source.

Alegerea materialului are un rol important asupra rezistenței și durabilității pieselor. Cel mai utilizat filament este PLA (*Polylactic Acid*), apreciat pentru ușurința la printare, însă relativ casant și sensibil la temperatură. O alternativă mai rezistentă este ABS (*Acrylonitrile Butadiene Styrene*), mai dur, dar mai dificil de printat și predispus la deformări. Pentru acest proiect a fost ales PETG (*Polyethylene Terephthalate Glycol*), care combină avantajele celor două, oferind o rezistență mecanică și o flexibilitate superioare PLA-ului, fără dificultățile de printare ale ABS-ului. Această rezistență este esențială pentru piesele unei mâini robotice, care sunt supuse în mod constant la tensiunile produse de tendoane și de elementele elastice.

Pe lângă alegerea materialului, calitatea pieselor depinde și de parametrii de printare, precum densitatea de umplere, numărul de straturi exterioare sau înălțimea unui strat. Documentarea privind aceste setări și ajustarea lor pentru piesele brațului robotic au necesitat mai multe încercări, fabricarea completă a componentelor însumând aproximativ 32 de ore de printare.

Capitolul 4. Analiză și fundamentare teoretică

Acest capitol prezintă fundamentele teoretice ale sistemului de control pentru mâna robotică, tratând fiecare componentă din lanțul de semnal din perspectivă analitică. Scopul este de a justifica deciziile arhitecturale prin prisma cerințelor de timp real, a preciziei numerice și a teoriei controlului optimal. Capitolul se concentrează pe principii și pe demonstrații matematice, fără a face referire la detalii specifice de implementare.

4.1. Arhitectura sistemului

Sistemul de control al mâinii robotice operează ca un lanț de procesare a semnalelor în buclă închisă. Fluxul principal de date, de la captura video până la actuarea mecanică, traversează cinci etape funcționale distincte, fiecare cu cerințe temporale bine definite.

4.1.1. Lanțul de semnal

Semnalul de comandă parcurge următoarele etape:

1. **Captură video și detecție** — Un sistem de viziune artificială (MediaPipe) procesează fluxul video de la o cameră și extrage coordonatele articulațiilor mâinii umane. Aceste coordonate sunt convertite în 8 unghiuri de servo, corespunzătoare celor 8 grade de libertate ale mâinii robotice.
2. **Transmisie serială** — Unghiurile sunt împachetate într-un cadru de 10 octeți și transmise prin UART la 115 200 baud către placa FPGA.
3. **Procesare FPGA** — Circuitul FPGA recepționează cadrul, validează suma de control, aplică filtrarea Kalman pe fiecare canal de servo și generează 8 semnale PWM corespunzătoare.
4. **Actuare** — Semnalele PWM, după izolarea galvanică și conversia de nivel, comandă cele 8 servomotoare ale mâinii robotice.
5. **Feedback** — Un convertor analog-digital pe 24 de biți citește tensiunile de feedback ale servomotoarelor, iar valorile măsurate sunt utilizate ca intrare în filtrul Kalman, închizând bucla de control.

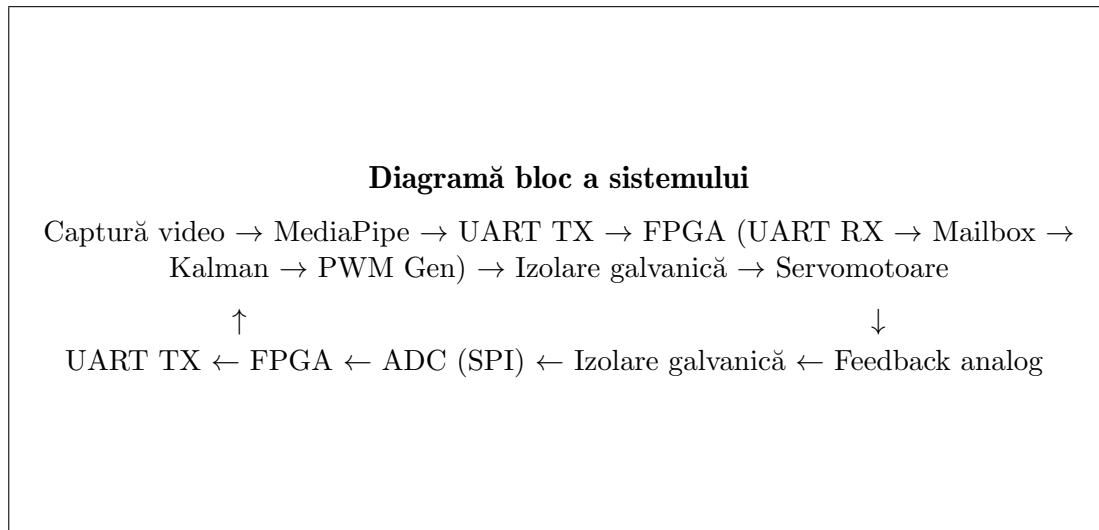


Figura 4.1: Diagrama bloc a lanțului de semnal, incluzând bucla de feedback prin ADC.

4.1.2. Ciclul de control de 20 ms

Elementul central al arhitecturii temporale este ciclul de control de 20 ms (50 Hz). Această frecvență nu este aleasă arbitrar, ci este dictată de specificația servomotoarelor standard:

- Servomotoarele de tip hobby așteaptă un semnal PWM cu o perioadă de 20 ms. Un impuls mai frecvent nu conferă un avantaj de precizie, iar unul mai rar poate duce la pierderea momentului de menținere.
- La o rată de actualizare de 50 Hz, intervalul de timp $\Delta t = 0,02$ s asigură o rezoluție temporală suficientă pentru mișcări umane (frecvența maximă a mișcărilor voluntare ale degetelor este de aproximativ 5–8 Hz [15]).
- Rata de 50 Hz oferă un ciclu de 2 000 000 de perioade de ceas la 100 MHz, furnizând un buget de timp generos pentru toate operațiunile de calcul.

Toate subsistemele funcționează sincronizat cu acest ciclu: recepția UART, prelucrarea Kalman, generarea PWM și citirea ADC se desfășoară în cadrul aceluiași interval de 20 ms.

4.2. Protocolul de comunicație UART

Comunicația dintre sistemul de viziune și FPGA se realizează prin interfața serială UART (*Universal Asynchronous Receiver/Transmitter*), configurată la 115 200 baud, 8 biți de date, fără paritate, 1 bit de stop (8N1).

4.2.1. Formatul cadrului

Fiecare cadru de comunicație conține 10 octeți, organizați conform Tabelului 4.1.

Tabela 4.1: Structura cadrului UART de 10 octeți.

Octet	Câmp	Descriere
0	Direcție	0xFF = scriere unghiuri, 0xFE = cerere de citire
1	Unghi servo 0	Valoare 0–180 (grade)
2	Unghi servo 1	Valoare 0–180 (grade)
3	Unghi servo 2	Valoare 0–180 (grade)
4	Unghi servo 3	Valoare 0–180 (grade)
5	Unghi servo 4	Valoare 0–180 (grade)
6	Unghi servo 5	Valoare 0–180 (grade)
7	Unghi servo 6	Valoare 0–180 (grade)
8	Unghi servo 7	Valoare 0–180 (grade)
9	Sumă de control	XOR al octeților 0–8

Câmpul de direcție servește drept delimitator de cadru și indicator de operație. Valoarea 0xFF indică o comandă de scriere — FPGA-ul trebuie să actualizeze unghiurile servomotoarelor cu valorile primite. Valoarea 0xFE indică o cerere de citire — FPGA-ul răspunde cu un cadru conținând unghiurile curente (filtrate) ale servomotoarelor.

4.2.2. Mecanismul sumei de control XOR

Suma de control asigură integritatea datelor transmise. Algoritmul funcționează după cum urmează:

La emițător: Se calculează suma de control ca operație XOR succesivă a tuturor octeților de date:

$$c = b_0 \oplus b_1 \oplus b_2 \oplus \dots \oplus b_8 \quad (4.1)$$

unde b_i reprezintă octetul i al cadrului, iar $\oplus$ este operația SAU-exclusiv pe biți. Valoarea c este plasată în octetul 9 al cadrului.

La receptor: Se calculează XOR-ul tuturor celor 10 octeți recepționați:

$$v = b_0 \oplus b_1 \oplus \dots \oplus b_8 \oplus b_9 \quad (4.2)$$

Substituind $b_9 = c = b_0 \oplus b_1 \oplus \dots \oplus b_8$, obținem:

$$v = (b_0 \oplus b_1 \oplus \dots \oplus b_8) \oplus (b_0 \oplus b_1 \oplus \dots \oplus b_8) = 0x00 \quad (4.3)$$

deoarece orice valoare XOR-ată cu ea însăși produce zero ($a \oplus a = 0, \forall a$). Dacă un singur bit este alterat în timpul transmisiei, rezultatul verificării va fi nenul, iar cadrul este respins.

Suma de control XOR detectează toate erorile de un singur bit și majoritatea erorilor multi-bit, fiind în același timp extrem de eficientă din punct de vedere computațional — necesită doar 9 operații XOR, fiecare executabilă într-un singur ciclu de ceas.

4.2.3. Analiza temporală a transmisiei

La rata de 115 200 baud cu formatul 8N1, fiecare octet necesită 10 biți pe linia serială (1 bit de start + 8 biți de date + 1 bit de stop). Timpul de transmisie pentru un cadru complet este:

$$T_{\text{cadru}} = \frac{10 \text{ octeți} \times 10 \text{ biți/octet}}{115\,200 \text{ biți/s}} = \frac{100}{115\,200} \approx 0,87 \text{ ms} \quad (4.4)$$

Acest timp reprezintă doar 4,3% din ciclul de control de 20 ms, lăsând un interval generos pentru procesare.

4.2.4. Sincronizarea ceasului UART

La un ceas de sistem de 100 MHz, numărul de perioade de ceas corespunzător unui bit UART este:

$$N_{\text{bit}} = \frac{f_{\text{clk}}}{f_{\text{baud}}} = \frac{100\,000\,000}{115\,200} \approx 868 \text{ perioade de ceas} \quad (4.5)$$

Eroarea de cuantizare este:

$$\varepsilon = \frac{868 \times 115\,200 - 100\,000\,000}{100\,000\,000} = \frac{16\,600}{100\,000\,000} \approx 0,017\% \quad (4.6)$$

Această eroare este neglijabilă — specificația UART tolerează abateri de până la $\pm 2\%$ de la rata de baud nominală.

Receptorul eșantionează fiecare bit la jumătatea perioadei acestuia, după $N_{\text{bit}}/2 = 434$ perioade de ceas de la frontul descrescător al bitului de start. Eșantionarea la mijlocul bitului maximizează marginea temporală față de tranzițiile de pe linia serială.

4.3. Generarea semnalelor PWM

Servomotoarele standard sunt comandate prin modulație în lățimea impulsului (*Pulse Width Modulation* — PWM). Această secțiune analizează relația matematică dintre unghiul dorit și parametrii semnalului PWM.

4.3.1. Specificația semnalului servo PWM

Un semnal PWM standard pentru servomotoare are următoarele caracteristici:

- **Perioada:** $T = 20 \text{ ms}$ (frecvența $f = 50 \text{ Hz}$)
- **Lățimea impulsului:** variabilă între $1,0 \text{ ms}$ (0°) și $2,0 \text{ ms}$ (180°)
- **Relația unghi–durată:** liniară

Relația dintre unghiul α (în grade) și durata impulsului τ (în milisecunde) este:

$$\tau(\alpha) = 1,0 + \frac{\alpha}{180} \times 1,0 = 1,0 + \frac{\alpha}{180} \quad [\text{ms}] \quad (4.7)$$

4.3.2. Cuantizarea la 100 MHz

La o frecvență de ceas de 100 MHz ($T_{\text{clk}} = 10 \text{ ns}$), valorile temporale se exprimă în număr de perioade de ceas:

Tabela 4.2: Corespondența dintre intervalele temporale și numărătoarele la 100 MHz.

Interval	Durată	Perioade de ceas
Perioadă completă	20 ms	2 000 000
Impuls minim (0°)	1,0 ms	100 000
Impuls maxim (180°)	2,0 ms	200 000
Interval activ	1,0 ms	100 000

Pragul de comparație (*threshold*) se calculează din unghiul α (valoare întreagă, $0 \leq \alpha \leq 180$):

$$\text{threshold}(\alpha) = 100\,000 + \alpha \times 556 \quad (4.8)$$

unde factorul de scală $556 \approx \lfloor 100\,000/180 \rfloor$ este obținut prin împărțirea intervalului activ de 100 000 de cicluri la cele 180 de grade.

4.3.3. Principiul de funcționare

Generatorul PWM utilizează un numărător liber care parcurge valorile de la 0 la 1 999 999 (corespunzând perioadei de 20 ms). Semnalul de ieșire este determinat de comparația:

$$\text{PWM}_{\text{out}} = \begin{cases} 1 & \text{dacă counter} < \text{threshold}(\alpha) \\ 0 & \text{altfel} \end{cases} \quad (4.9)$$

Rezoluția unghiulară rezultată este:

$$\Delta\alpha = \frac{180^\circ}{100\,000} = 0,0018^\circ \quad (4.10)$$

Această rezoluție depășește cu mult capacitatea mecanică a servomotoarelor (tipic $\pm 1^\circ$), asigurând că precizia sistemului digital nu constituie un factor limitativ.

4.4. Filtrul Kalman — fundamentare matematică

Filtrul Kalman reprezintă nucleul algoritmic al sistemului. Această secțiune derivă pas cu pas filtrul Kalman cu două stări utilizat pentru estimarea optimală a poziției și vitezei fiecărui servomotor.

4.4.1. Formularea problemei

Fiecare servomotor primește o comandă de poziție prin PWM, dar poziția reală este afectată de:

- Inerția mecanică a servomotorului și a articulației;
- Sarcina variabilă (greutatea degetului, forțe externe);
- Zgomotul electronic al senzorului de feedback.

Se dorește estimarea optimală a poziției reale x_0 și a vitezei x_1 , combinând predicția bazată pe un model fizic cu măsurătoarea de la convertorul analog-digital.

4.4.2. Modelul de stare

Vectorul de stare conține poziția și viteza:

$$\mathbf{x} = \begin{bmatrix} x_0 \\ x_1 \end{bmatrix} = \begin{bmatrix} \text{poziție [grade]} \\ \text{viteză [grade/s]} \end{bmatrix} \quad (4.11)$$

Ecuția de tranziție a stării

Modelul cinematic presupune viteză constantă pe durata unui interval de eșantionare Δt :

$$x_0(k+1) = x_0(k) + \Delta t \cdot x_1(k) \quad (4.12)$$

$$x_1(k+1) = x_1(k) \quad (4.13)$$

Aceste ecuații se exprimă matriceal prin intermediul matricei de tranziție $\mathbf{F}$:

$$\mathbf{x}(k+1) = \mathbf{F} \cdot \mathbf{x}(k) + \mathbf{w}(k) \quad (4.14)$$

unde:

$$\mathbf{F} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}, \quad \Delta t = 0,02 \text{ s} \quad (4.15)$$

iar $\mathbf{w}(k) \sim \mathcal{N}(\mathbf{0}, \mathbf{Q})$ este zgomotul de proces, modelat ca o variabilă aleatoare gaussiană cu medie zero și matrice de covarianță:

$$\mathbf{Q} = \begin{bmatrix} Q_{00} & 0 \\ 0 & Q_{11} \end{bmatrix} \quad (4.16)$$

Matricea $\mathbf{Q}$ este diagonală, ceea ce presupune că perturbațiile în poziție și viteză sunt necorelate. Elementul Q_{00} (în grade^2) cuantifică incertitudinea predicției de poziție, iar Q_{11} (în $(\text{grade/s})^2$) cuantifică incertitudinea predicției de viteză.

Ecuția de observare

Senzorul de feedback (convertorul analog-digital) măsoară doar poziția:

$$z(k) = \mathbf{H} \cdot \mathbf{x}(k) + v(k) \quad (4.17)$$

unde:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (4.18)$$

iar $v(k) \sim \mathcal{N}(0, R)$ este zgomotul de măsurătoare, cu varianța R (în grade^2). Matricea $\mathbf{H}$ extrage doar componenta de poziție din vectorul de stare — viteza nu este direct observabilă.

Matricea de covarianță a erorii de estimare

Incetitudinea estimării este descrisă de matricea de covarianță simetrică $\mathbf{P}$:

$$\mathbf{P} = \begin{bmatrix} P_{00} & P_{01} \\ P_{01} & P_{11} \end{bmatrix} \quad (4.19)$$

unde P_{00} este varianța erorii de poziție, P_{11} este varianța erorii de viteză, iar P_{01} este covarianța încrucișată (corelația dintre erorile de poziție și viteză).

4.4.3. Pasul de predicție

La fiecare ciclu de control, filtrul Kalman începe cu o predicție a stării pe baza modelului cinematic. Acest pas *propagă* starea și incertitudinea înainte în timp.

Predicția stării

Aplicând ecuația de tranziție (4.14) fără termenul de zgomot:

$$\hat{\mathbf{x}}^- = \mathbf{F} \cdot \hat{\mathbf{x}} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{x}_1 \end{bmatrix} = \begin{bmatrix} \hat{x}_0 + \Delta t \cdot \hat{x}_1 \\ \hat{x}_1 \end{bmatrix} \quad (4.20)$$

Interpretare fizică:

- Poziția predicționată: $\hat{x}_0^- = \hat{x}_0 + \Delta t \cdot \hat{x}_1$ — poziția anterioară plus distanța parcursă la viteza estimată pe durata Δt .
- Viteza predicționată: $\hat{x}_1^- = \hat{x}_1$ — viteza rămâne neschimbată în etapa de predicție, conform ipotezei de viteză constantă.

De exemplu, dacă servomotorul se află la 90° cu o viteză estimată de $100^\circ/\text{s}$, după $\Delta t = 0,02$ s predicția este $90 + 0,02 \times 100 = 92^\circ$.

Predicția covarianței

Matricea de covarianță predicționată se obține din:

$$\mathbf{P}^- = \mathbf{F} \cdot \mathbf{P} \cdot \mathbf{F}^\top + \mathbf{Q} \quad (4.21)$$

Dezvoltăm produsul matriceal $\mathbf{F} \cdot \mathbf{P} \cdot \mathbf{F}^\top$ explicit. Mai întâi, calculăm $\mathbf{F} \cdot \mathbf{P}$:

$$\mathbf{F} \cdot \mathbf{P} = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} P_{00} & P_{01} \\ P_{01} & P_{11} \end{bmatrix} = \begin{bmatrix} P_{00} + \Delta t \cdot P_{01} & P_{01} + \Delta t \cdot P_{11} \\ P_{01} & P_{11} \end{bmatrix} \quad (4.22)$$

Apoi, înmulțim la dreapta cu $\mathbf{F}^\top$:

$$\mathbf{F}^\top = \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \quad (4.23)$$

$$\mathbf{F} \cdot \mathbf{P} \cdot \mathbf{F}^\top = \begin{bmatrix} P_{00} + \Delta t \cdot P_{01} & P_{01} + \Delta t \cdot P_{11} \\ P_{01} & P_{11} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \Delta t & 1 \end{bmatrix} \quad (4.24)$$

Calculând element cu element și adăugând $\mathbf{Q}$, obținem componentele matricei de covarianță predicționată:

$$P_{00}^- = P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + Q_{00} \quad (4.25)$$

$$P_{01}^- = P_{01} + \Delta t \cdot P_{11} \quad (4.26)$$

$$P_{11}^- = P_{11} + Q_{11} \quad (4.27)$$

Interpretare fizică:

- Ecuția (4.25): Incertitudinea în poziție crește datorită propagării incertitudinii de viteză ($\Delta t^2 \cdot P_{11}$), a corelației existente ($2\Delta t \cdot P_{01}$) și a zgomotului de proces (Q_{00}). Termenul dominant este cel legat de viteză — o viteză incertă produce o predicție de poziție incertă.
- Ecuția (4.26): Covarianța încrucișată crește cu $\Delta t \cdot P_{11}$, reflectând faptul că integrarea unei viteze incerte introduce corelație între erorile de poziție și viteză.
- Ecuția (4.27): Incertitudinea în viteză crește doar prin adunarea zgomotului de proces Q_{11} , deoarece modelul presupune viteză constantă.

Fără pasul de actualizare, covarianța ar crește nelimitat la fiecare iterație — aceasta este degradarea naturală a oricărei predicții bazate pe un model imperfect.

4.4.4. Pasul de actualizare (corecție)

Pasul de actualizare încorporează măsurătoarea de la ADC pentru a corecta predicția. Aceasta este esența filtrului Kalman: combinarea optimală a informației din predicție și din măsurătoare.

Inovația

Inovația (*innovation*) reprezintă diferența dintre măsurătoare și predicție:

$$y = z - \mathbf{H} \cdot \hat{\mathbf{x}}^- = z - \hat{x}_0^- \quad (4.28)$$

unde z este poziția măsurată de ADC. Inovația cuantifică *surpriza* — cât de mult diferă realitatea (măsurată) de așteptare (predicția). O inovație mare indică fie o predicție proastă, fie o măsurătoare zgomotoasă.

Covarianța inovației

Covarianța inovației combină incertitudinea predicției cu zgomotul de măsurătoare:

$$S = \mathbf{H} \cdot \mathbf{P}^- \cdot \mathbf{H}^T + R \quad (4.29)$$

Deoarece $\mathbf{H} = \begin{bmatrix} 1 & 0 \end{bmatrix}$, extragerea se simplifică la:

$$S = P_{00}^- + R \quad (4.30)$$

Valoarea S reprezintă incertitudinea totală asupra inovației — suma incertitudinii predicției de poziție și a zgomotului senzorului.

Câștigul Kalman

Câștigul Kalman (*Kalman gain*) determină ponderea acordată măsurătorii în corecția estimării:

$$\mathbf{K} = \mathbf{P}^- \cdot \mathbf{H}^T \cdot S^{-1} \quad (4.31)$$

Dezvoltând:

$$\mathbf{K} = \begin{bmatrix} P_{00}^- \\ P_{01}^- \end{bmatrix} \cdot \frac{1}{S} = \begin{bmatrix} K_0 \\ K_1 \end{bmatrix} \quad (4.32)$$

unde:

$$K_0 = \frac{P_{00}^-}{S} = \frac{P_{00}^-}{P_{00}^- + R} \quad (4.33)$$

$$K_1 = \frac{P_{01}^-}{S} = \frac{P_{01}^-}{P_{00}^- + R} \quad (4.34)$$

Analiza limitelor câștigului K_0 revelă comportamentul adaptiv al filtrului:

- Dacă $P_{00}^- \gg R$ (incertitudine mare a predicției, senzor precis): $K_0 \rightarrow 1$, filtrul „are încredere” în măsurătoare.
- Dacă $P_{00}^- \ll R$ (predicție precisă, senzor zgomotos): $K_0 \rightarrow 0$, filtrul „are încredere” în predicție.
- În regim staționar, câștigul converge la o valoare intermediară care realizează compromisul optim.

Actualizarea stării

Starea corectată se obține adăugând corecția proporțională cu inovația:

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^- + \mathbf{K} \cdot \mathbf{y} \quad (4.35)$$

Componentă cu componentă:

$$\hat{x}_0 = \hat{x}_0^- + K_0 \cdot y \quad (4.36)$$

$$\hat{x}_1 = \hat{x}_1^- + K_1 \cdot y \quad (4.37)$$

Ecuția (4.36) corectează poziția estimată: dacă măsurătoarea indică o poziție mai mare decât cea predicționată ($y > 0$), poziția estimată crește cu $K_0 \cdot y$.

Ecuția (4.37) este deosebit de semnificativă: **viteza este corectată pe baza erorii de poziție**, deși viteza nu este direct măsurată. Mecanismul este următorul: dacă măsurătorile arată în mod consistent că servomotorul se deplasează mai departe decât predicția ($y > 0$ repetat), câștigul K_1 produce o corecție pozitivă a vitezei, iar estimarea de viteză crește treptat.

Aceasta este o proprietate fundamentală a filtrului Kalman: prin corelația încrucișată P_{01} (cuprinsă în K_1), informația din măsurătoarea de poziție se propagă în estimarea de viteză. Filtrul „învață” viteza reală, chiar dacă modelul presupune viteză constantă.

Actualizarea covarianței

După încorporarea măsurătorii, incertitudinea estimării scade:

$$\mathbf{P} = (\mathbf{I} - \mathbf{K} \cdot \mathbf{H}) \cdot \mathbf{P}^- \quad (4.38)$$

Dezvoltăm matriceal:

$$\mathbf{I} - \mathbf{K} \cdot \mathbf{H} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} K_0 \\ K_1 \end{bmatrix} \begin{bmatrix} 1 & 0 \end{bmatrix} = \begin{bmatrix} 1 - K_0 & 0 \\ -K_1 & 1 \end{bmatrix} \quad (4.39)$$

Înmulțind cu $\mathbf{P}^-$:

$$\mathbf{P} = \begin{bmatrix} 1 - K_0 & 0 \\ -K_1 & 1 \end{bmatrix} \begin{bmatrix} P_{00}^- & P_{01}^- \\ P_{01}^- & P_{11}^- \end{bmatrix} \quad (4.40)$$

Componentele rezultante sunt:

$$P_{00} = (1 - K_0) \cdot P_{00}^- \quad (4.41)$$

$$P_{01} = (1 - K_0) \cdot P_{01}^- \quad (4.42)$$

$$P_{11} = -K_1 \cdot P_{01}^- + P_{11}^- \quad (4.43)$$

Deoarece $0 < K_0 < 1$, factorul $(1 - K_0)$ reduce P_{00} și P_{01} — fiecare măsurătoare diminuează incertitudinea estimării. Ecuația (4.43) arată că și incertitudinea vitezei scade (prin termenul $-K_1 \cdot P_{01}^-$), deși mai lent, deoarece viteza este doar indirect observată.

4.4.5. Ciclu predicție-actualizare: o perspectivă unitară

Algoritmul complet se rezumă la cele două etape, repetate la fiecare 20 ms:

Tabela 4.3: Rezumatul algoritmului Kalman cu două stări.

Pas	Ecuație	Semnificație
<i>Predicție</i>		
1	$\hat{x}_0^- = \hat{x}_0 + \Delta t \cdot \hat{x}_1$	Propagare poziție
2	$\hat{x}_1^- = \hat{x}_1$	Propagare viteză
3	$P_{00}^- = P_{00} + 2\Delta t \cdot P_{01} + \Delta t^2 \cdot P_{11} + Q_{00}$	Creștere incertitudine
4	$P_{01}^- = P_{01} + \Delta t \cdot P_{11}$	
5	$P_{11}^- = P_{11} + Q_{11}$	
<i>Actualizare</i>		
6	$y = z - \hat{x}_0^-$	Inovație
7	$S = P_{00}^- + R$	Covarianță inovație
8	$K_0 = P_{00}^-/S, \quad K_1 = P_{01}^-/S$	Câștig Kalman
9	$\hat{x}_0 = \hat{x}_0^- + K_0 \cdot y$	Corecție poziție
10	$\hat{x}_1 = \hat{x}_1^- + K_1 \cdot y$	Corecție viteză
11	$P_{00} = (1 - K_0) \cdot P_{00}^-$	Reducere incertitudine
12	$P_{01} = (1 - K_0) \cdot P_{01}^-$	
13	$P_{11} = -K_1 \cdot P_{01}^- + P_{11}^-$	

Echilibrul dinamic predicție-actualizare asigură funcționarea filtrului: predicția crește incertitudinea (covarianța crește), iar actualizarea o reduce (covarianța scade). În regim staționar, cele două efecte se echilibrează, iar câștigul Kalman converge la o valoare fixă.

4.4.6. Alegerea parametrilor și semnificația lor fizică

Performanța filtrului Kalman este determinată de alegerea matricelor de zgomot $\mathbf{Q}$ și $\mathbf{R}$. Aceste valori nu au soluții analitice universale — ele codifică asumțiile proiectantului despre mediul de operare.

Zgomotul de proces Q

Matricea Q modelează gradul în care realitatea deviază de la modelul de viteză constantă. Pentru un servomotor:

- Q_{00} (varianța perturbației de poziție): o valoare mică (de exemplu, $0,01 \text{ grade}^2$) indică faptul că poziția nu se schimbă semnificativ din cauze nemodulate de viteză. Valori tipice se situează între $0,001$ și $0,1 \text{ grade}^2$.
- Q_{11} (varianța perturbației de viteză): o valoare mai mare (de exemplu, $0,1 \text{ (grade/s)}^2$) permite filtrului să „urmărească” schimbările de viteză. Valori tipice se situează între $0,01$ și $1,0 \text{ (grade/s)}^2$.

Efect: Un Q mic produce un răspuns mai neted dar mai lent — filtrul „are încredere” în predicție și reacționează lent la schimbările reale. Un Q mare produce un răspuns mai rapid dar mai zgomotos — filtrul urmărește fidel măsurătoarea, dar amplifică și zgomotul.

Zgomotul de măsurătoare R

Scalarul R (în grade^2) cuantifică încrederea în măsurătoarea ADC. Acesta depinde de:

- Rezoluția ADC-ului și zgomotul de cuantizare;
- Zgomotul analogic al circuitului de feedback (inductanțe parazite, interferențe electromagnetice de la motoare);
- Neliniaritățile potențiometrului de feedback al servomotorului.

Efect: Un R mic (de exemplu, $0,1 \text{ grade}^2$) face filtrul responsiv la măsurători, dar vulnerabil la zgomotul senzorului. Un R mare (de exemplu, 10 grade^2) produce o filtrare mai agresivă, dar întârzie răspunsul la schimbări reale.

Incertitudinea inițială P_0

Matricea de covarianță inițială trebuie să reflecte lipsa de cunoaștere la pornire:

$$P_0 = \begin{bmatrix} P_{\text{init}} & 0 \\ 0 & P_{\text{init}} \end{bmatrix} \quad (4.44)$$

O valoare mare a P_{init} (de exemplu, 10) semnalizează filtrului că estimarea inițială este nesigură. Consecința practică: în primele iterații, câștigul Kalman este apropiat de 1 , iar filtrul „urmărește” măsurătoarea aproape integral. Pe măsură ce covarianța converge (după 10 – 20 de iterații, adică $0,2$ – $0,4$ s), filtrul ajunge la comportamentul staționar și nu mai depinde de valoarea inițială.

Compromisul fundamental

Relația dintre parametrii de reglare poate fi sintetizată astfel:

Tabela 4.4: Efectul parametrilor de reglare asupra comportamentului filtrului.

Configurație	Răspuns	Netezie
Q mic, R mare	Lent	Neted
Q mare, R mic	Rapid	Zgomotos
$Q \approx R$	Echilibrat	Moderat

Alegerea optimă depinde de aplicație: pentru o mână robotică care trebuie să urmărească mișcări umane rapide, se preferă un compromis cu răspuns relativ rapid, acceptând un nivel moderat de zgomot care este absorbit de inerția mecanică a servomotoarelor.

4.5. Aritmetica Q16.16 în virgulă fixă

Circuitele FPGA nu dispun în general de unități de calcul în virgulă mobilă. Prin urmare, toate calculele filtrului Kalman trebuie efectuate în aritmetică cu virgulă fixă. Această secțiune prezintă formatul Q16.16 și justifică alegerea acestuia.

4.5.1. Formatul de reprezentare

Formatul Q16.16 utilizează un cuvânt de 32 de biți cu semn, în care:

- **Biții 31:16** (16 biți) — partea întreagă, inclusiv bitul de semn (complement față de 2);
- **Biții 15:0** (16 biți) — partea fracționară.

Reprezentarea internă a unui număr real v este:

$$v_{Q16.16} = \lfloor v \times 2^{16} \rfloor = \lfloor v \times 65\,536 \rfloor \quad (4.45)$$

Rezoluția (cel mai mic increment reprezentabil) este:

$$\varepsilon = 2^{-16} = \frac{1}{65\,536} \approx 1,53 \times 10^{-5} \quad (4.46)$$

Domeniul de reprezentare este:

$$-2^{15} \leq v < 2^{15} - 2^{-16}, \quad \text{adică} \quad -32\,768 \leq v < 32\,767,999985 \quad (4.47)$$

4.5.2. Exemple de conversie

Tabelul 4.5 prezintă conversia câtorva valori relevante pentru filtrul Kalman.

Tabela 4.5: Exemple de reprezentare în format Q16.16.

Valoare reală	Calcul	Reprezentare Q16.16
1,0	$1,0 \times 65\,536$	65 536
0,02 (Δt)	$0,02 \times 65\,536$	1 311
0,01 (Q_{00})	$0,01 \times 65\,536$	655
0,1 (Q_{11})	$0,1 \times 65\,536$	6 554
10,0 (P_{init})	$10,0 \times 65\,536$	655 360
180,0	$180,0 \times 65\,536$	11 796 480

4.5.3. Operații aritmetice

Adunarea și scăderea

Adunarea și scăderea a două numere Q16.16 se realizează direct, fără operații suplimentare:

$$(a + b)_{Q16.16} = a_{Q16.16} + b_{Q16.16} \quad (4.48)$$

Aceasta funcționează deoarece ambii operanzi au același factor de scală 2^{16} .

Înmulțirea

Înmulțirea a două numere Q16.16 produce un rezultat pe 64 de biți cu 32 de biți fracționari:

$$a_{Q16.16} \times b_{Q16.16} = (a \times 2^{16}) \times (b \times 2^{16}) = a \times b \times 2^{32} \quad (4.49)$$

Pentru a reveni la formatul Q16.16 (factor de scală 2^{16}), rezultatul pe 64 de biți trebuie deplasat la dreapta cu 16 biți:

$$(a \times b)_{Q16.16} = \frac{a_{Q16.16} \times b_{Q16.16}}{2^{16}} = (a_{Q16.16} \times b_{Q16.16}) \gg 16 \quad (4.50)$$

Echivalent, se extrag biții [47 : 16] din produsul pe 64 de biți. Biții [15 : 0] sunt biți fracționari suplimentari (precizie pierdută prin trunchiere), iar biții [63 : 48] sunt extensia de semn.

Împărțirea

Pentru a menține precizia la împărțire, deîmpărțitul trebuie pre-deplasat la stânga cu 16 biți:

$$\left(\frac{a}{b}\right)_{Q16.16} = \frac{a_{Q16.16} \ll 16}{b_{Q16.16}} \quad (4.51)$$

Această pre-deplasare compensează faptul că împărțirea întreagă ar pierde partea fracționară. Operandul $a_{Q16.16} \ll 16$ necesită o reprezentare pe 48 de biți pentru a evita depășirea.

4.5.4. Justificarea alegerii Q16.16 față de Q8.8

Un format mai compact, Q8.8 (16 biți, 8 biți întregi + 8 biți fracționari), ar fi insuficient. Domeniul întreg al formatului Q8.8 cu semn este $[-128, 127]$. Reprezentarea valorii maxime de unghi:

$$180_{Q8.8} = 180 \times 2^8 = 180 \times 256 = 46\,080 \quad (4.52)$$

Aceasta depășește valoarea maximă a unui întreg pe 16 biți cu semn ($2^{15} - 1 = 32\,767$), provocând o depășire (*overflow*).

În schimb, formatul Q16.16:

$$180_{Q16.16} = 180 \times 65\,536 = 11\,796\,480 \quad (4.53)$$

Această valoare se încadrează confortabil în domeniul unui întreg pe 32 de biți cu semn ($2^{31} - 1 = 2\,147\,483\,647$).

De asemenea, rezoluția fracționară a formatului Q8.8 este $2^{-8} = 0,00391$, ceea ce ar introduce erori de cuantizare semnificative în calculele covarianței, unde valori precum $Q_{00} = 0,01$ ar fi reprezentate cu o eroare relativă de $\sim 2,5\%$. Formatul Q16.16, cu rezoluția de $2^{-16} \approx 0,0000153$, reduce această eroare la $\sim 0,001\%$.

4.6. Interfața SPI și convertorul AD7124-8

Bucula de feedback a sistemului utilizează un convertor analog-digital de înaltă rezoluție conectat prin interfața serială sincronă SPI (*Serial Peripheral Interface*).

4.6.1. Protocolul SPI — Modul 3

Interfața SPI definește patru semnale: SCLK (ceas), MOSI (date master $\rightarrow$ slave), MISO (date slave $\rightarrow$ master) și CS (selecție chip, activ pe nivel logic „0”). Există patru moduri de operare, definite de polaritatea ceasului (CPOL) și faza datelor (CPHA).

Convertorul AD7124-8 operează în **Modul 3** (CPOL = 1, CPHA = 1):

Tabela 4.6: Caracteristicile SPI Mod 3.

Parametru	Valoare
CPOL (Clock Polarity)	1 — SCLK în repaus la nivel logic „1”
CPHA (Clock Phase)	1 — datele sunt valide pe frontul crescător al SCLK
Modificare date	Pe frontul descrescător al SCLK
Eșantionare date	Pe frontul crescător al SCLK

Transferul se realizează cu bitul cel mai semnificativ (MSB) primul. Frecvența SCLK este derivată din ceasul de sistem prin divizare:

$$f_{\text{SCLK}} = \frac{f_{\text{clk}}}{N_{\text{div}}} = \frac{100 \text{ MHz}}{20} = 5 \text{ MHz} \quad (4.54)$$

Această frecvență respectă limita maximă specificată de convertorul AD7124-8 și oferă un transfer de 8 biți în $8 \times 200 \text{ ns} = 1,6 \mu\text{s}$.

4.6.2. Convertorul AD7124-8

AD7124-8 este un convertor analog-digital de tip sigma-delta ($\Sigma\Delta$) cu următoarele caracteristici relevante:

- **Rezoluție:** 24 de biți, oferind o gamă dinamică de $2^{24} = 16\,777\,216$ niveluri de cuantizare;
- **Canale:** 8 canale analogice independente, câte unul pentru fiecare servomotor;
- **Mod de conversie:** conversie continuă (*continuous conversion mode*), în care convertorul efectuează conversii succesive fără intervenția masterului;
- **Filtru digital:** sinc³, cu rata de ieșire configurabilă la 50 Hz (corespunzând ciclului de control de 20 ms).

4.6.3. Modul DATA_STATUS

În modul DATA_STATUS, fiecare citire a registrului de date returnează 4 octeți în loc de 3:

Tabela 4.7: Formatul citirii în modul DATA_STATUS.

Octet	Conținut	Descriere
0	DATA[23:16]	Octetul superior al conversiei
1	DATA[15:8]	Octetul median al conversiei
2	DATA[7:0]	Octetul inferior al conversiei
3	STATUS	Registru de stare: biții 3:0 = ID canal

Identificatorul de canal (biții [3 : 0] din octetul de stare) permite masterului să determine automat cărei intrări analogice îi corespunde conversia curentă, eliminând necesitatea secvenționării manuale a canalelor.

4.6.4. Conversia ADC $\rightarrow$ unghi

Valoarea brută pe 24 de biți, în configurație unipolară ($0 \leq \text{adc_raw} < 2^{24}$), trebuie convertită în unghiul de poziție al servomotorului ($0-180^\circ$). Conversia liniară este:

$$\alpha = \frac{\text{adc_raw} \times 180}{2^{24}} \quad (4.55)$$

Această operație se implementează eficient prin:

$$\alpha = (\text{adc_raw} \times 180) \gg 24 \quad (4.56)$$

unde $\gg 24$ denotă deplasarea la dreapta cu 24 de biți, echivalentă cu împărțirea întreagă la 2^{24} . Produsul $\text{adc_raw} \times 180$ necesită cel mult $24+8 = 32$ de biți, încadrându-se într-un registru standard.

Rezoluția unghiulară a conversiei ADC este:

$$\Delta\alpha_{\text{ADC}} = \frac{180}{2^{24}} \approx 1,07 \times 10^{-5^\circ} \quad (4.57)$$

Această rezoluție depășește cu mai multe ordine de mărime precizia mecanică a servomotoarelor, astfel încât cuantizarea ADC nu constituie o sursă semnificativă de eroare.

4.7. Bugetul de timp

Această secțiune analizează cantitativ fezabilitatea prelucrării tuturor celor 8 canale de servo în cadrul unui singur ciclu de control de 20 ms.

4.7.1. Costul computațional per canal

Algoritmul Kalman derivat în Secțiunea 4.4 necesită, pentru fiecare canal de servo, următoarele operații aritmetice:

Tabela 4.8: Inventarul operațiilor aritmetice per canal de servo.

Etapa	Înmulțiri	Împărțiri	Adunări/scăderi
Predicție stare (ec. 4.20)	1	0	1
Predicție covarianță (ec. 4.25–4.27)	3	0	5
Inovație (ec. 5.9)	0	0	1
Covarianță inovație (ec. 4.29)	0	0	1
Câștig Kalman (ec. 4.33–4.34)	0	2	0
Actualizare stare (ec. 4.36–4.37)	2	0	2
Actualizare covarianță (ec. 4.41–4.43)	3	0	4
Conversie ieșire (Q16.16 → unghi)	0	0	0
Încărcare/stocare RAM	5	0	0
Total	14	2	14

Estimarea numărului de cicluri de ceas presupune:

- Înmulțirea Q16.16: ~ 4 cicluri (incluzând acumularea pe 64 de biți și deplasarea);
- Împărțirea Q16.16: ~ 35 cicluri (împărțire secvențială pe 48 de biți);
- Adunarea/scăderea: 1 ciclu.

Costul total per canal:

$$C_{\text{canal}} = 14 \times 4 + 2 \times 35 + 14 \times 1 = 56 + 70 + 14 = 140 \text{ cicluri} \quad (4.58)$$

Adăugând cicluri auxiliare (încărcare registre, secvențiere FSM, latență de acces la RAM), estimarea crește la aproximativ ~ 182 de cicluri per canal.

4.7.2. Bugetul total pentru 8 canale

$$C_{\text{total}} = 8 \times 182 = 1\,456 \text{ cicluri de ceas} \quad (4.59)$$

Bugetul de timp disponibil per ciclu de control:

$$N_{\text{disponibil}} = f_{\text{clk}} \times T_{\text{ciclu}} = 100 \times 10^6 \times 0,02 = 2\,000\,000 \text{ cicluri} \quad (4.60)$$

Gradul de utilizare:

$$U = \frac{C_{\text{total}}}{N_{\text{disponibil}}} = \frac{1\,456}{2\,000\,000} = 0,073\% \approx 0,07\% \quad (4.61)$$

4.7.3. Marja de siguranță și extensibilitate

Gradul de utilizare de 0,07% lasă o marjă enormă — peste 99,9% din ciclurile de ceas disponibile sunt neutilizate. Această marjă permite:

1. **Extinderea la filtru Kalman cu 3 stări:** Adăugarea accelerației ca a treia variabilă de stare crește matricea $\mathbf{P}$ de la 2×2 la 3×3 (6 elemente unice, datorită simetriei), iar costul computațional crește la aproximativ 45 de înmulțiri și 3 împărțiri per canal. Chiar în acest caz, totalul pentru 8 canale rămâne sub 5 000 de cicluri — încă neglijabil față de bugetul de 2 000 000.
2. **Adăugarea de funcții suplimentare:** Generatoare de rampe de accelerare, limitatoare de curent, sau algoritmi de detectare a coliziunilor ar putea fi integrate fără

a compromite respectarea ciclului de 20 ms.

3. **Reducerea frecvenței de ceas:** Dacă consumul de putere devine o constrângere, ceasul ar putea fi redus substanțial (de exemplu, la 10 MHz) menținând în continuare un buget de timp suficient (200 000 cicluri vs. $\sim 1\,500$ necesare).

Această analiză confirmă că abordarea de prelucrare secvențială (un canal la un moment dat, multiplexat în timp) este viabilă și nu necesită paralelizare hardware, simplificând semnificativ arhitectura și reducând consumul de resurse FPGA.

Fundamentele teoretice prezentate în acest capitol — protocolul de comunicație, algoritmul de generare PWM, filtrul Kalman cu demonstrație completă, aritmetica în virgulă fixă, interfața cu convertorul analog-digital și analiza bugetului de timp — constituie baza pentru proiectarea detaliată și implementarea hardware descrise în capitolul următor.

Capitolul 5. Proiectare de detaliu și implementare

Acest capitol prezintă proiectarea detaliată și implementarea în VHDL a sistemului de control FPGA pentru mâna robotică. Fiecare modul hardware este descris din perspectiva funcționalității, a interfeței de porturi, a mașinii de stări interne și a deciziilor de proiectare adoptate. Codul sursă complet este inclus în anexe; în acest capitol se prezintă doar fragmente relevante, însoțite de explicații funcționale.

5.1. Schema generală a sistemului

Sistemul este compus din 12 module VHDL interconectate ierarhic prin intermediul modului de nivel superior `basys3_top`. Figura 5.1 prezintă diagrama bloc a întregului sistem, evidențiind fluxul de date de la intrarea UART până la ieșirile PWM.

```
uart_rx → frame_rx → mailbox → source_mux → kalman_engine → pwm_gen (8
                                     instanțe)
                                     system_tick_gen — ceas 20 ms pentru toate modulele
                                     demo_rom → source_mux (intrare alternativă)
                                     spi_master ← ad7124_ctrl → kalman_engine (feedback ADC)
                                     frame_tx → uart_tx (răspuns către PC)
```

Figura 5.1: Diagrama bloc a sistemului — fluxul de interconectare al modulelor VHDL

Cele 12 module sunt enumerate în Tabelul 5.1, împreună cu rolul funcțional al fiecăruia.

Tabela 5.1: Lista modulelor VHDL și rolul lor în sistem

Modul	Rol funcțional
<code>system_tick_gen</code>	Generează pulsul periodic de 20 ms
<code>uart_rx</code>	Recepția serială UART la 115 200 baud
<code>uart_tx</code>	Transmisia serială UART la 115 200 baud
<code>frame_rx</code>	Parser de cadre de 10 octeți cu verificare checksum
<code>frame_tx</code>	Serializare cadru de răspuns (direcție 0xFE)
<code>mailbox</code>	Buffer dublu (ping-pong) pentru decuplare temporală
<code>source_mux</code>	Multiplexor combinațional: UART vs. demo
<code>demo_rom</code>	ROM cu 21 de cadre-cheie pentru mod demonstrativ
<code>spi_master</code>	Master SPI mod 3, 5 MHz
<code>ad7124_ctrl</code>	Controler pentru ADC-ul AD7124-8
<code>kalman_engine</code>	Filtru Kalman cu 2 stări, Q16.16, 8 canale
<code>pwm_gen</code>	Generator PWM pentru un servo (8 instanțe)

Interconexiunile dintre module respectă principiul fluxului de date unidirecțional: datele curg de la recepția UART către generarea PWM, cu o ramură de feedback de la ADC prin filtrul Kalman. Semnalul de ceas de 20 ms (`tick_20ms`) sincronizează toate modulele, asigurând că actualizarea servomotoarelor se face cu o perioadă fixă, independentă de rata de sosire a cadrelor UART.

5.2. Modulul `system_tick_gen`

Modulul `system_tick_gen` generează un puls de sincronizare cu perioada exactă de 20 ms, necesară pentru ciclul standard al servomotoarelor (50 Hz). Implementarea se bazează pe un numărător pe 21 de biți care numără de la 0 la 1 999 999, corespunzând celor 2 000 000 de cicluri de ceas la 100 MHz necesare pentru 20 ms.

5.2.1. Interfața de porturi

Tabela 5.2: Porturile modulului `system_tick_gen`

Port	Direcție	Descriere
<code>clk</code>	intrare	Ceasul de sistem, 100 MHz
<code>rst</code>	intrare	Reset sincron
<code>tick_20ms</code>	ieșire	Puls de un ciclu de ceas la fiecare 20 ms
<code>tick_cnt</code>	ieșire	Valoarea curentă a numărătorului (21 biți)

5.2.2. Funcționare

Numărătorul `counter` este de tip `unsigned(20 downto 0)` și se incrementează la fiecare front crescător al ceasului. Când atinge valoarea constantei `PERIOD = 1 999 999`, se resetează la zero. Ieșirea `tick_20ms` este '1' exclusiv în ciclul de ceas în care numărătorul egalează `PERIOD`, generând astfel un puls de exactmente un ciclu de ceas (10 ns) la fiecare 20 ms.

Ieșirea `tick_cnt` furnizează valoarea curentă a numărătorului, utilizată de modulul `pwm_gen` pentru compararea cu pragul PWM. Deoarece ambele module partajează același numărător de bază, semnalele PWM sunt perfect sincronizate cu ciclul de 20 ms.

Calcul: $100 \text{ MHz} \times 20 \text{ ms} = 100 \times 10^6 \times 20 \times 10^{-3} = 2\,000\,000$ cicluri. Numărătorul numără $0, 1, \dots, 1\,999\,999$, deci necesită $\lceil \log_2(2\,000\,000) \rceil = 21$ biți.

5.3. Modulul `uart_rx` — Recepția UART

Modulul `uart_rx` implementează un receptor UART complet la 115 200 baud, cu protecție contra metastabilității și eșantionare la mijlocul bitului. Configurația este 8N1 (8 biți de date, fără paritate, 1 bit de stop).

5.3.1. Calculul parametrilor de baud

Rata de ceas a sistemului este de 100 MHz, iar rata de transfer (baud rate) este de 115 200 baud. Numărul de cicluri de ceas pe bit este:

$$\text{CLKS_PER_BIT} = \frac{100\,000\,000}{115\,200} - 1 = 868 - 1 = 867 \quad (5.1)$$

Scăderea cu 1 se datorează faptului că numărătorul pornește de la 0. Pentru eșantionarea la mijlocul bitului, se utilizează constanta:

$$\text{HALF_BIT} = \left\lfloor \frac{868}{2} \right\rfloor - 1 = 433 \quad (5.2)$$

Eșantionarea la mijlocul bitului este esențială deoarece semnalul serial prezintă tranziții la granițele biților, iar valorile sunt stabile doar în zona centrală. Prin așteptarea a 433 de cicluri după detectarea frontului descrescător al bitului de start, receptorul se poziționează în centrul fiecărui bit, maximizând marja de toleranță la variațiile de frecvență între emițător și receptor.

5.3.2. Protecția contra metastabilității

Semnalul `rx_serial` provine dintr-un domeniu de ceas extern (linia UART a PC-ului) și este asincron față de ceasul FPGA de 100 MHz. Când un semnal asincron este eșantionat de un flip-flop, există o probabilitate nenulă ca acesta să fie capturat exact în momentul tranziției, rezultând o stare metastabilă — o tensiune intermediară între '0' și '1' care nu se rezolvă în timpul de setup al flip-flop-ului. Această situație poate provoca comportament nedeterminist în logica din aval.

Soluția standard constă în utilizarea a două flip-flop-uri în cascadă, cunoscute sub numele de *synchronizer* sau *double-FF*:

```

1 signal rx_sync1 : std_logic := '1';
2 signal rx_sync2 : std_logic := '1';
3
4 process(clk)
5 begin
6     if rising_edge(clk) then
7         rx_sync1 <= rx_serial;
8         rx_sync2 <= rx_sync1;
9     end if;
10 end process;
```

Listing 5.1: Sincronizator cu dublu flip-flop pentru protecția contra metastabilității

Primul flip-flop (`rx_sync1`) poate intra în stare metastabilă, dar are un ciclu de ceas complet (10 ns la 100 MHz) pentru a se rezolva înainte ca al doilea flip-flop (`rx_sync2`) să-l eșantioneze. Probabilitatea ca metastabilitatea să se propage prin ambele flip-flop-uri este neglijabilă — de ordinul 10^{-20} erori pe secundă pentru flip-flop-urile din tehnologia Artix-7.

Valorile inițiale sunt '1' deoarece linia UART este inactivă la nivel logic înalt (*idle high*).

5.3.3. Mașina de stări

Receptorul UART utilizează o mașină de stări cu 4 stări:

1. **S_IDLE** — Stare de așteptare. Monitorizează linia `rx_sync2` pentru detectarea frontului descrescător care marchează începutul bitului de start.
2. **S_START** — Verificarea bitului de start. Numărătorul de ceas (`clk_count`) avansează până la `HALF_BIT` (433), moment în care se verifică dacă linia este încă '0'. Dacă da, tranziția a fost legitimă și se trece la recepția datelor; dacă nu, a fost un impuls parazit (*glitch*) și se revine la **S_IDLE**.

3. **S_DATA** — Recepția celor 8 biți de date. La fiecare `CLKS_PER_BIT` (867) cicluri, valoarea de pe linie este memorată în registrul de deplasare la poziția indicată de `bit_idx`. Biții sunt recepționați LSB first, conform standardului UART. După bitul 7, se trece la `S_STOP`.
4. **S_STOP** — Așteptarea bitului de stop. Se așteaptă `CLKS_PER_BIT` cicluri, timp în care linia trebuie să fie la nivel logic '1'. La finalul perioadei, `rx_valid` este activat pentru un ciclu de ceas, iar `rx_data` conține octetul recepționat complet.

Durata totală de recepție a unui octet este: 1 bit start + 8 biți date + 1 bit stop = 10 biți $\times$ 868 cicluri = 8680 cicluri $\approx$ 86,8 μ s, corespunzând exact unui caracter la 115200 baud.

5.4. Modulul `uart_tx` — Transmisia UART

Modulul `uart_tx` este echivalentul simetric al receptorului, realizând transmisia serială la 115200 baud. La activarea semnalului `tx_start`, octetul prezent pe portul `tx_data` este salvat în registrul intern `data_reg`, iar transmisia începe.

5.4.1. Mașina de stări

Transmițătorul utilizează aceleași 4 stări ca receptorul:

1. **S_IDLE** — Linia de transmisie este menținută la '1' (idle). La detectarea semnalului `tx_start`, octetul este salvat și transmisia începe.
2. **S_START** — Linia este forțată la '0' pentru durata unui bit complet (868 cicluri de ceas), semnalizând receptorului începutul unui caracter.
3. **S_DATA** — Cei 8 biți de date sunt transmiși secvențial, câte un bit la fiecare 868 cicluri, începând cu bitul cel mai puțin semnificativ (LSB first). Semnalul `tx_serial` ia valoarea bitului curent din `data_reg(bit_idx)`.
4. **S_STOP** — Linia revine la '1' pentru durata unui bit, marcând bitul de stop. La final, mașina de stări revine la `S_IDLE`.

Semnalul `tx_busy` indică faptul că transmițătorul este ocupat (orice stare diferită de `S_IDLE`). Modulele din amonte (în particular `frame_tx`) verifică acest semnal înainte de a iniția o nouă transmisie, prevenind suprapunerea caracterelor.

Spre deosebire de receptor, transmițătorul nu necesită protecție contra metastabilității, deoarece atât datele de intrare cât și semnalul de ieșire se află în același domeniu de ceas de 100 MHz.

5.5. Modulul `frame_rx` — Parserul de cadre

Modulul `frame_rx` assemblează cadre de 10 octeți din octeții individuali recepționați de `uart_rx`. Protocolul de cadre definit în Capitolul 4 specifică formatul:

Octet 0	Octeții 1–8	Octet 9
Direcție (0xFF/0xFE)	8 unghiuri servo (0–180)	Checksum XOR

5.5.1. Mașina de stări

Parserul de cadre utilizează 3 stări:

1. **S_WAIT_DIR** — Așteaptă un octet de direcție valid (0xFF pentru scriere unghiuri, 0xFE pentru citire). Orice alt octet este ignorat, oferind o resincronizare naturală a protocolului. La primirea unui octet valid, acesta este salvat în `dir_reg` și este inițializat acumulatorul XOR (`xor_acc`) cu valoarea direcției.

2. **S_RECV_DATA** — Colectează 8 octeți de date (unghiurile servomotoarelor). Fiecare octet recepționat este stocat în registrul **angles_reg** la poziția corespunzătoare și este acumulat în **xor_acc** prin operația XOR. Numărătorul **byte_count** urmărește câți octeți au fost primiți. După al 8-lea octet, se trece la S_RECV_CHKSUM.
3. **S_RECV_CHKSUM** — Recepționează octetul de checksum. Verificarea se face prin calculul $\text{xor_acc} \oplus \text{rx_data}$: dacă rezultatul este 0x00, cadrul este valid și semnalul **frame_valid** este activat pentru un ciclu de ceas; în caz contrar, **checksum_err** este activat. În ambele cazuri, mașina revine la S_WAIT_DIR.

5.5.2. Mecanismul de timeout

Dacă un cadru nu este completat în 1 ms (100 000 cicluri de ceas), modulul revine automat la starea S_WAIT_DIR. Acest mecanism previne blocarea în cazul pierderii de octeți pe linia serială. Numărătorul de timeout (**timeout_cnt**) pe 17 biți se incrementează în fiecare ciclu de ceas când starea nu este S_WAIT_DIR și se resetează la zero la fiecare octet recepționat valid. Valoarea pragului este **TIMEOUT_MAX** = 99 999 (≈ 1 ms la 100 MHz).

5.5.3. Principiul verificării XOR

Emitătorul calculează checksum-ul ca $\bigoplus_{i=0}^8 b_i$ (XOR-ul tuturor octeților de la 0 la 8) și îl plasează în octetul 9. Receptorul acumulează XOR-ul tuturor celor 10 octeți: dacă niciun bit nu a fost alterat în transmisie, rezultatul este $\bigoplus_{i=0}^9 b_i = \bigoplus_{i=0}^8 b_i \oplus b_9 = x \oplus x = 0$. Orice bit inversat în tranzit produce un rezultat nenul, iar cadrul este rejectat.

5.6. Modulul *frame_tx* — Transmisia cadrelor de răspuns

Modulul *frame_tx* serializează un cadru de răspuns atunci când calculatorul gazdă trimite un cadru cu direcția 0xFE (cerere de citire). Răspunsul conține cele 8 unghiuri filtrate curent, precedate de octetul de direcție 0xFE și urmate de checksum-ul XOR.

5.6.1. Mașina de stări

Transmisia se desfășoară în 7 stări, grupate în 3 faze:

1. **S_IDLE** — Așteaptă activarea semnalului **send_start**. La activare, unghiurile sunt salvate în **angles_reg**.
2. **S_SEND_DIR / S_WAIT_DIR** — Transmite octetul de direcție 0xFE prin **uart_tx**. Inițializează acumulatorul XOR cu 0xFE. Așteaptă ca **tx_busy** să devină '0' înainte de a continua.
3. **S_SEND_DATA / S_WAIT_DATA** — Transmite secvențial cei 8 octeți de unghiuri, acumulând checksum-ul XOR. Indexul **byte_idx** contorizează octeții transmiși, de la 0 la 7.
4. **S_SEND_CHKSUM / S_WAIT_CHKSUM** — Transmite octetul de checksum (**xor_acc**) și revine la S_IDLE.

Semnalul **frame_busy** indică faptul că transmisia unui cadru este în curs. Modulul de nivel superior verifică acest semnal pentru a preveni inițierea simultană a mai multor cadre de răspuns.

Între fiecare octet transmis, modulul verifică semnalul **tx_busy** de la **uart_tx** pentru a se asigura că transmițătorul serial a terminat de trimis octetul precedent. La 115 200 baud, transmisia unui octet complet (10 biți) durează aproximativ 86,8 μ s, iar un cadru complet de 10 octeți durează aproximativ 868 μ s.

5.7. Modulul mailbox — Buffer dublu

Modulul `mailbox` implementează un mecanism de decuplare temporală între rata de sosire a cadrelor UART (variabilă, dependentă de procesarea MediaPipe pe calculatorul gazdă) și ciclul fix de 20 ms al servomotoarelor.

5.7.1. Principiul ping-pong

Modulul conține două registre de 64 de biți:

- `buf_write` — registrul de scriere, actualizat oricând sosește un cadru UART valid.
- `buf_read` — registrul de citire, actualizat exclusiv pe frontul semnalului `tick_20ms`.

Un semnal de stare `new_pending` indică dacă un cadru nou a fost depus de la ultimul tick de 20 ms. La fiecare tick, dacă `new_pending` este activ, conținutul `buf_write` este copiat în `buf_read`, iar `new_pending` este resetat. Ieșirea `rd_new` pulsează pentru un ciclu de ceas la fiecare actualizare.

Acest mecanism oferă două garanții importante:

1. **Menținerea poziției:** Dacă niciun cadru nou nu sosește într-un ciclu de 20 ms, servomotoarele mențin ultima poziție comandată. Mâna robotică rămâne stabilă chiar dacă comunicarea UART este temporar întreruptă.
2. **Rata de eșantionare:** Dacă mai multe cadre UART sosesc în același interval de 20 ms, doar ultimul este utilizat. Aceasta este corectă deoarece doar cel mai recent unghi comandat este relevant pentru ciclul curent.

Condiția de scriere a mailbox-ului în modulul de nivel superior este: `mb_wr_valid <= frame_valid and (not frame_chksum_err)`, asigurând că doar cadrele cu checksum valid sunt acceptate.

5.8. Modulul source_mux — Multiplexor de surse

Modulul `source_mux` este un multiplexor pur combinațional care selectează sursa unghiurilor țintă pentru filtrul Kalman:

```
1 target_angles <= demo_angles when demo_mode = '1'
2           else mailbox_angles;
```

Listing 5.2: Implementarea multiplexorului de surse

Când comutatorul `sw0` de pe placa Basys3 este în poziția ON (`demo_mode = '1'`), unghiurile provin din modulul `demo_rom`. În caz contrar, unghiurile provin din `mailbox`, deci din cadrele UART recepționate de la calculatorul gazdă.

Logica este pur combinațională, fără registre, producând o latență de propagare de ordinul nanosecundelor. Această decizie de proiectare este adecvată deoarece semnalul `sw0` se modifică rar (doar la acțiunea manuală a utilizatorului) și nu necesită filtrare de debounce — un comutator basculant nu prezintă fenomenul de bouncing, spre deosebire de un buton.

5.9. Modulul demo_rom — Secvențe demonstrative

Modulul `demo_rom` conține o memorie ROM cu 21 de cadre-cheie (*keyframes*) care definesc secvențe de mișcare predefinite ale mâinii robotice. Fiecare cadru-cheie conține 8 unghiuri (câte un octet per servo, totalul de 64 biți) și o durată de menținere exprimată în tick-uri de 20 ms.

5.9.1. Formatul cadru-cheie

Fiecare intrare în ROM este definită ca un **record** VHDL:

```

1 type keyframe_t is record
2   angles : std_logic_vector(63 downto 0);
3   hold    : unsigned(7 downto 0);
4 end record;
```

Listing 5.3: Tipul record pentru un cadru-cheie din ROM

Câmpul **hold** specifică numărul de tick-uri de 20 ms cât se mențin unghiurile respective. De exemplu, **hold** = 50 corespunde unei durate de $50 \times 20 \text{ ms} = 1 \text{ secundă}$.

5.9.2. Secvențele de mișcare

ROM-ul conține următoarele secvențe, ordonate astfel:

Tabela 5.3: Secvențele demonstrative stocate în ROM

Indici	Secvența	Durată
0–3	Deschidere/închidere pumn ($2\times$)	4 s
4–8	Fluturare (rotire încheietură stânga/dreapta)	2,5 s
9–13	Flexii individuale degete	3 s
14–15	Gest <i>thumbs up</i>	2,5 s
16–17	Gest <i>pointing</i>	2,5 s
18–20	Flexia/extensia cotului	3 s

Constanta **LAST_KF** = 20 definește ultimul cadru-cheie valid. La depășirea acestui indice, numărătorul de cadre-cheie **kf_index** se resetează la zero, creând un ciclu continuu de demonstrație.

5.9.3. Logica de avansare

Un numărător **hold_counter** se incrementează la fiecare tick de 20 ms. Când atinge valoarea **hold** a cadrului-cheie curent, indexul **kf_index** avansează la următorul cadru-cheie și **hold_counter** se resetează. Dezactivarea intrării **enable** (prin comutatorul **sw0**) resetează atât indexul cât și numărătorul, astfel încât reactivarea pornește secvența de la început.

Ordinea servomotoarelor în vectorul de 64 biți este: degetul mic, degetul inelar, degetul mijlociu, degetul arătător, mișcarea palmară a degetului mare, tensiunea degetului mare, rotația încheieturii și extensia cotului. Convențional, valoarea 0 corespunde poziției complet închise (pentru degete) sau complet flexate (pentru cot), iar 180 corespunde poziției complet deschise, respectiv extinse.

5.10. Modulul *spi_master* — Interfața SPI

Modulul *spi_master* implementează un master SPI generic care operează în modul 3 (CPOL=1, CPHA=1), conform cerințelor ADC-ului AD7124-8.

5.10.1. Parametrii SPI mod 3

În modul 3:

- SCLK este inactiv la nivel logic '1' (CPOL=1).
- Datele sunt modificate pe frontul descrescător al SCLK (*falling edge*) și eșantionate pe frontul crescător (*rising edge*) — CPHA=1.
- Frecvența SCLK rezultă din divizarea ceasului de sistem: $100 \text{ MHz} / 20 = 5 \text{ MHz}$. Constanta CLK_DIV = 9 generează o semiperioadă de 10 cicluri (numărare de la 0 la 9), rezultând o perioadă completă de 20 cicluri de ceas.

5.10.2. Mașina de stări

1. **S_IDLE** — SCLK este menținut la '1'. La activarea **start**, octetul de transmis este salvat în **shift_out**, iar bitul MSB este plasat pe linia MOSI. Semnalul **cs_n** este activat la '0'.
2. **S_LEADING_EDGE** — Numărătorul de divizare **div_cnt** avansează de la 0 la CLK_DIV (9). La atingerea pragului, SCLK comută la '0' (front descrescător), moment în care receptorul eșantionează MOSI. Se trece la S_TRAILING_EDGE.
3. **S_TRAILING_EDGE** — Numărătorul avansează din nou până la 9. La atingerea pragului, SCLK comută la '1' (front crescător), moment în care masterul eșantionează MISO. Bitul recepționat este introdus în **shift_in**. Dacă toți 8 biții au fost transferați (**bit_cnt** = 7), se trece la S_DONE; altfel, se avansează **bit_cnt** și se actualizează MOSI cu următorul bit.
4. **S_DONE** — Octetul recepționat este copiat pe **rx_data**, semnalul **done** pulsează, iar mașina revine la S_IDLE.

5.10.3. Protecția contra metastabilității pe MISO

Similar receptorului UART, linia MISO provine dintr-un circuit extern (ADC-ul AD7124-8) și este asincronă față de ceasul FPGA. Două flip-flop-uri în cascadă (**miso_s1**, **miso_s2**) sincronizează semnalul înainte de eșantionare:

```

1 process (clk)
2 begin
3     if rising_edge(clk) then
4         miso_s1 <= miso;
5         miso_s2 <= miso_s1;
6     end if;
7 end process;
```

Listing 5.4: Sincronizator pe linia MISO

Ordinea de transfer este MSB first (bitul cel mai semnificativ este trimis primul), conform convenției AD7124-8.

5.11. Modulul ad7124_ctrl — Controlerul ADC

Modulul **ad7124_ctrl** controlează ADC-ul AD7124-8 pe 24 de biți prin intermediul modulului **spi_master**. Funcționarea se desfășoară în două faze: inițializarea (o singură dată după reset) și citirea ciclică continuă.

5.11.1. Faza de inițializare

Secvența de inițializare cuprinde următorii pași:

1. **Reset hardware:** Se transmit 8 octeți de 0xFF prin SPI. Conform documentației AD7124-8, transmiterea a cel puțin 64 de biți consecutivi de '1' resetează registrele

interne ale ADC-ului la valorile implicite.

2. **Așteptare post-reset:** Un interval de 4 ms (400 000 cicluri de ceas) permite ADC-ului să finalizeze procedura internă de resetare. Această valoare depășește specificația minimă din fișa de catalog, asigurând o marjă confortabilă.
3. **Configurare registre:** Se scriu secvențial 11 registre de configurare, fiecare conștinând dintr-un octet de comandă (adresa registrului) urmat de 2 sau 3 octeți de date:

Tabela 5.4: Secvența de configurare a registrelor AD7124-8

Registru	Valoare	Semnificație
ADC_Control (0x01)	0x0DC0	Full power, conversie continuă, DATA_STATUS activat, referință internă
Config_0 (0x19)	0x0070	Unipolar, buffere intrare, referință internă, PGA=1
Filter_0 (0x21)	0x060180	Filtru sinc3, FS=384, ODR=50 Hz
Channel 0 (0x09)	0x8011	Activat, Setup 0, AINP=0, AINM=AVSS
Channel 1 (0x0A)	0x8031	Activat, Setup 0, AINP=1, AINM=AVSS
Channel 2 (0x0B)	0x8051	Activat, Setup 0, AINP=2, AINM=AVSS
Channel 3 (0x0C)	0x8071	Activat, Setup 0, AINP=3, AINM=AVSS
Channel 4 (0x0D)	0x8091	Activat, Setup 0, AINP=4, AINM=AVSS
Channel 5 (0x0E)	0x80B1	Activat, Setup 0, AINP=5, AINM=AVSS
Channel 6 (0x0F)	0x80D1	Activat, Setup 0, AINP=6, AINM=AVSS
Channel 7 (0x10)	0x80F1	Activat, Setup 0, AINP=7, AINM=AVSS

Rata de ieșire a datelor (ODR) de 50 Hz corespunde exact ciclului de servo de 20 ms, asigurând că o nouă măsurătoare este disponibilă la fiecare actualizare a servomotoarelor.

Toate cele 8 canale sunt configurate în mod *single-ended*, cu intrarea negativă conectată la AVSS (masa analogică). Fiecare canal citește semnalul de feedback analog de la potențiometrul intern al servomotorului MG996R corespunzător.

5.11.2. Faza de citire ciclică

După finalizarea inițializării, modulul intră într-un ciclu continuu de citire a registrului de date. Secvența de citire este:

1. Se trimite comanda de citire a registrului de date: 0x42 (RD=1 | adresa registrului de date = 0x02).
2. Se recepționează 3 octeți de date (24 biți, valoarea ADC brută) și 1 octet de status (care conține, în biții 3:0, numărul canalului de pe care provine conversiunea).

3. Se extrage numărul canalului din octetul de status și se efectuează conversia în unghi:

$$\text{angle} = \frac{\text{adc_raw} \times 180}{2^{24}} \quad (5.3)$$

Implementarea în VHDL utilizează înmulțirea cu 180 urmată de deplasare la dreapta cu 24 de biți (echivalent cu împărțirea la 2^{24}). Aceasta mapează domeniul complet al ADC-ului (0x000000 – 0xFFFFFFF) la intervalul 0° – 180° .

Un registru de 8 biți `channels_read` urmărește care canale au fost citite. Când toate cele 8 bituri sunt setate (0xFF), modulul emite semnalul `meas_valid` și furnizează toți cei 8 unghiuri mășurați pe portul `meas_angles`, apoi reșetează registrul pentru un nou ciclu de citire.

Semnalul `config_done` este conectat la LED-ul 0 de pe placa Basys3, oferind o indicație vizuală că inițializarea ADC-ului s-a finalizat cu succes.

5.12. Modulul `kalman_engine` — Filtrul Kalman

Modulul `kalman_engine` implementează nucleul algoritmului de filtrare Kalman cu 2 stări (poziție și viteză) în aritmetică virgulă fixă Q16.16, multiplexat în timp pe cele 8 canale de servo. Acesta este modulul central al proiectului, responsabil pentru stabilizarea mișcărilor mâinii robotice prin estimarea optimă a traiectoriei pe baza măsurătorilor zgomotoase.

5.12.1. Constantele de configurare

Tabelul 5.5 prezintă constantele utilizate de filtru, cu valorile în reprezentare Q16.16 și echivalentele reale.

Tabela 5.5: Constantele filtrului Kalman în reprezentare Q16.16

Constantă	Valoare Q16.16	Valoare reală	Semnificație
ONE_Q16	65 536	1,0	Reprezentarea unității
DT_Q16	1 311	0,02 s	Perioada de actualizare (20 ms)
Q00	655	0,01 grad ²	Zgomot de proces — poziție
Q11	6 554	0,1 (grad/s) ²	Zgomot de proces — viteză
R_NOISE	65 536	1,0 grad ²	Zgomot de măsurare
P_INIT	655 360	10,0	Incertitudine inițială

Valorile Q00 și Q11 definesc cât de mult *distrust-ăm* predicția (valori mai mari = predicție mai nesigură, filtru mai receptiv). R_NOISE definește cât de mult *distrust-ăm* măsurătoarea ADC (valori mai mari = măsurătoare mai zgomotoasă, filtru mai lin). Aceste constante pot fi ajustate în funcție de comportamentul observat pe hardware-ul real.

5.12.2. Memoria de stare (State RAM)

Starea filtrului Kalman pentru cele 8 canale este stocată într-o memorie RAM internă de $8 \times 5 = 40$ cuvinte de câte 32 de biți:

Tabela 5.6: Structura memoriei de stare pentru un canal

Offset	Variabilă	Descriere
+0	x_0	Poziția estimată (grade, Q16.16)
+1	x_1	Viteza estimată (grade/secundă, Q16.16)
+2	P_{00}	Incertitudinea poziției
+3	P_{01}	Corelația poziție-viteză
+4	P_{11}	Incertitudinea vitezei

Adresa de bază pentru canalul k este $5k$, iar funcția `ch_base` calculează această adresă. La inițializare, x_0 și x_1 sunt setate la zero, iar P_{00} și P_{11} sunt setate la `P_INIT` = 10,0 (incertitudine inițială mare, semnificând că filtrul nu are încă informații despre poziția reală a servomotoarelor).

5.12.3. Funcția de multiplicare Q16.16

Multiplicarea a două numere Q16.16 produce un rezultat pe 64 de biți cu 32 de biți fracționari. Pentru a reveni la formatul Q16.16 (16 biți fracționari), se extrag biții 47:16 din produsul complet:

```

1 function mul_q16(a : signed(31 downto 0);
2               b : signed(31 downto 0))
3     return signed is
4     variable product : signed(63 downto 0);
5 begin
6     product := a * b;
7     return product(47 downto 16);
8 end function;
```

Listing 5.5: Funcția de multiplicare în virgulă fixă Q16.16

Această operație este echivalentă matematic cu $(a \times b) \gg 16$, menținând precizia fracționară fără a necesita unitate de virgulă mobilă. Sintetizatorul Vivado mapează aceste multiplicări pe blocurile DSP48E1 disponibile în Artix-7, eliminând necesitatea implementării cu logică combinațională.

5.12.4. Mașina de stări a filtrului Kalman

Filtrul procesează secvențial cele 8 canale de servo, executând pentru fiecare canal un ciclu complet de predicție și actualizare. Mașina de stări cuprinde 15 stări, grupate funcțional:

Inițializare (`S_INIT_RAM`)

La pornire sau după reset, se parcurge memoria de stare de la adresa 0 la 39, inițializând fiecare cuvânt conform structurii din Tabelul 5.6. Pozițiile de offset 2 și 4 (P_{00} și P_{11}) primesc valoarea `P_INIT`, iar celelalte sunt inițializate la zero. După finalizare, semnalul `initialized` este activat și mașina trece la `S_IDLE`.

Încărcare (S_LOAD)

Se citesc cele 5 variabile de stare ale canalului curent din RAM și se convertește unghiul măsurat din format pe 8 biți în Q16.16 (multiplicare cu 65536). Funcția `get_angle_q16` extrage octetul corespunzător canalului curent din vectorul de 64 biți al măsurătorilor.

Predicție (S_PREDICT_X, S_PREDICT_P00, P01, P11)

Se aplică ecuațiile de predicție ale filtrului Kalman:

$$x_{0,\text{pred}} = x_0 + dt \cdot x_1 \quad (5.4)$$

$$x_{1,\text{pred}} = x_1 \quad (5.5)$$

Viteza nu se modifică în predicție deoarece modelul cu 2 stări nu include accelerația. Predicția covarianței crește incertitudinea:

$$P'_{00} = P_{00} + 2 \cdot dt \cdot P_{01} + dt^2 \cdot P_{11} + Q_{00} \quad (5.6)$$

$$P'_{01} = P_{01} + dt \cdot P_{11} \quad (5.7)$$

$$P'_{11} = P_{11} + Q_{11} \quad (5.8)$$

Implementarea VHDL a predicției P_{00} utilizează funcția `mul_q16` pentru toate produsele:

```

1 when S_PREDICT_P00 =>
2   pp00 <= p00 + mul_q16(DT_Q16, p01) +
3     mul_q16(DT_Q16, p01) +
4     mul_q16(DT_Q16, mul_q16(DT_Q16, p11)) + Q00;
5 fsm <= S_PREDICT_P01;
```

Listing 5.6: Predicția covarianței P_{00} în VHDL

Actualizare (S_UPDATE_S, K0, K1, X, P)

Inovația (*innovation*) reprezintă diferența dintre măsurătoarea reală și predicție:

$$y = z_{\text{meas}} - x_{0,\text{pred}} \quad (5.9)$$

Covarianța inovației combină incertitudinea predicției cu zgomotul de măsurare:

$$S = P'_{00} + R \quad (5.10)$$

Câștigul Kalman determină ponderea acordată măsurătorii vs. predicției:

$$K_0 = P'_{00}/S \quad (5.11)$$

$$K_1 = P'_{01}/S \quad (5.12)$$

Dacă incertitudinea predicției (P'_{00}) este mare comparativ cu zgomotul de măsurare (R), câștigul se apropie de 1,0 — filtrul încredințează mai mult măsurătorii. Dacă P'_{00} este mic, câștigul se apropie de 0 — filtrul încredințează mai mult predicția.

Implementarea diviziunii în VHDL utilizează operatorul / aplicat pe valori extinse la 64 de biți, cu deplasare la stânga prealabilă a numărătorului cu 16 biți pentru a menține precizia Q16.16:

```

1 when S_UPDATE_K0 =>
2   if s_val /= 0 then
3     tmp64 := shift_left(resize(pp00, 64), 16);
4     k0 <= resize(tmp64 / resize(s_val, 64), 32);
5   else
6     k0 <= ONE_Q16;
7   end if;

```

Listing 5.7: Calculul câștigului Kalman K0 cu protecție împotriva împărțirii la zero

Protecția împotriva împărțirii la zero setează $K_0 = 1,0$ în cazul extrem în care $S = 0$, ceea ce ar însemna încredere totală în măsurătoare.

Corecția stării:

$$x_0 = x_{0,\text{pred}} + K_0 \cdot y \quad (5.13)$$

$$x_1 = x_{1,\text{pred}} + K_1 \cdot y \quad (5.14)$$

Ecuția (5.14) este mecanismul prin care viteza *învață*: dacă măsurătoarea arată constant că servomotorul s-a deplasat mai mult decât a prezis predicția, inovația y este pozitivă, iar $K_1 \cdot y$ crește estimarea vitezei. Reciproc, dacă servomotorul este în repaus, inovația scade, iar viteza converge spre zero.

Actualizarea covarianței (incertitudinea scade după integrarea măsurătorii):

$$P_{00} = (1 - K_0) \cdot P'_{00} \quad (5.15)$$

$$P_{01} = (1 - K_0) \cdot P'_{01} \quad (5.16)$$

$$P_{11} = -K_1 \cdot P'_{01} + P'_{11} \quad (5.17)$$

Stocare și avansare (S_STORE, S_NEXT_CH, S_OUTPUT)

Cele 5 variabile de stare actualizate sunt scrise înapoi în RAM la adresa de bază a canalului curent. Poziția estimată (x_0) este convertită de la Q16.16 la un unghi pe 8 biți prin deplasare la dreapta cu 16 biți, urmată de limitare la intervalul 0–180:

```

1 result_angle := to_integer(shift_right(x0, 16));
2 if result_angle < 0 then
3   result_angle := 0;
4 elsif result_angle > 180 then
5   result_angle := 180;
6 end if;

```

Listing 5.8: Conversia Q16.16 la unghi pe 8 biți cu limitare

După procesarea tuturor celor 8 canale ($\text{channel} = 0, 1, \dots, 7$), starea S_OUTPUT transferă toți unghiurile filtrate pe portul `output_angles` și activează semnalul `output_valid` pentru un ciclu de ceas. Mașina revine apoi la S_IDLE, așteptând următorul tick de 20 ms.

5.12.5. Buget temporal

Fiecare canal necesită aproximativ 14 multiplicări, 2 diviziuni și 14 adunări, totalizând circa 182 cicluri de ceas. Procesarea tuturor celor 8 canale necesită aproximativ $8 \times 182 = 1456$ cicluri din cele 2 000 000 disponibile între două tick-uri de 20 ms, reprezentând un grad de utilizare de doar 0,07%. Aceasta lasă un buget de ceas generos pentru eventuale extinderi viitoare, cum ar fi trecerea la un filtru Kalman cu 3 stări.

5.13. Modulul `pwm_gen` — Generatorul PWM

Modulul `pwm_gen` generează un semnal PWM standard pentru un servomotor. Standardul servo definește:

- Perioadă: 20 ms (50 Hz)
- Puls de 1,0 ms $\rightarrow$ 0° (poziție minimă)
- Puls de 2,0 ms $\rightarrow$ 180° (poziție maximă)

La frecvența de ceas de 100 MHz:

- 1,0 ms = 100 000 de cicluri
- 2,0 ms = 200 000 de cicluri
- Intervalul util = 100 000 de cicluri pentru 180°
- Factor de conversie: $100\,000/180 \approx 556$

Pragul PWM se calculează ca:

$$\text{threshold} = 100\,000 + \text{angle} \times 556 \quad (5.18)$$

Ieșirea PWM este '1' cât timp numărătorul intern (pe 21 de biți, numărând de la 0 la 1 999 999) este mai mic decât pragul, și '0' în rest. Acest principiu de comparație este implementat printr-o singură instrucțiune combinațională:

```
1 pwm_out <= '1' when counter < threshold else '0';
```

Listing 5.9: Generarea semnalului PWM prin comparație

5.13.1. Generarea a 8 instanțe

În modulul de nivel superior `basys3_top`, se generează 8 instanțe ale modulului `pwm_gen` prin intermediul instrucțiunii `for...generate`:

```
1 gen_pwm : for i in 0 to 7 generate
2     u_pwm : entity work.pwm_gen
3         port map (
4             clk      => clk,
5             rst      => rst,
6             angle    => kalman_out((7-i)*8+7 downto (7-i)*8),
7             pwm_out  => pwm_out(i)
8         );
9 end generate;
```

Listing 5.10: Generarea a 8 instanțe PWM prin instrucțiunea generate

Fiecare instanță primește octetul de unghi corespunzător din vectorul de 64 biți al ieșirii filtrului Kalman. Indexarea $(7-i) \times 8$ asigură corespondența corectă între canalele logice (0–7) și pozițiile din vectorul de biți.

Deoarece fiecare instanță PWM posedă propriul numărător hardware, toate cele 8 canale funcționează în paralel, fără multiplexare în timp. Această proprietate a implementării FPGA garantează că actualizarea unui canal nu afectează în niciun fel celelalte canale, eliminând jitter-ul de interleaving prezent în implementările bazate pe microcontroller.

5.14. Modulul *basys3_top* — Integrarea la nivel superior

Modulul *basys3_top* este entitatea de nivel superior care instanțiază și interconectează toate cele 12 module descrise anterior. Porturile sale corespund pinilor fizici ai plăcii Basys3 Artix-7.

5.14.1. Conectarea porturilor

Tabela 5.7: Maparea principală a porturilor la funcții

Port	Pin Basys3	Funcție
clk	W5	Oscilator 100 MHz
sw0	V17	Comutator mod demo
btnC	U18	Buton reset (centru)
RsRx	B18	UART recepție
RsTx	A18	UART transmisie
JA0–JA3	J1, L2, J2, G2	PWM servouri 0–3 (pinky, ring, middle, index)
JB0–JB3	A14, A16, B15, B16	PWM servouri 4–7 (thumb, wrist, elbow)
JC0 (CS)	K17	SPI chip select AD7124
JC1 (MOSI)	M18	SPI date ieșire
JC2 (MISO)	N17	SPI date intrare
JC3 (SCLK)	P18	SPI ceas
led[0:15]	diverse	Indicatoare de stare

5.14.2. Gestionarea pinilor neutilizați

Toți pinii neutilizați ai plăcii Basys3 sunt configurați la stări sigure pentru a preveni intrări flotante și consum inutil de curent:

- **JA4–JA7, JB4–JB7, JC4–JC7:** Jumătatea superioară a conectorilor Pmod, forțată la '0' (drive low).
- **seg[6:0], dp, an[3:0]:** Afișajul cu 7 segmente este activ la nivel logic scăzut; se forțează la '1' pentru a dezactiva toate segmentele și cifrele.
- **vgaRed, vgaGreen, vgaBlue, Hsync, Vsync:** Ieșirile VGA sunt forțate la '0'.
- **PS2Clk, PS2Data:** Interfața PS/2 este lăsată în înaltă impedanță ('Z') deoarece este un bus open-drain.
- **JXADC[7:0]:** Header-ul analogic este lăsat în înaltă impedanță ('Z').
- **sw[15:1], btnU/D/L/R:** Comutatoarele și butoanele neutilizate rămân ca intrări, citite dar neasignate.

5.14.3. Indicatoarele LED

Cele 16 LED-uri de pe placa Basys3 sunt utilizate ca indicatoare de stare:

Tabela 5.8: Asignarea indicatoarelor LED

LED	Semnal indicat
0	Configurare ADC finalizată (<code>adc_cfg_done</code>)
1	Măsurătoare ADC validă (<code>meas_valid</code>)
2	Cadru UART recepționat (<code>frame_valid</code>)
3	Eroare de checksum (<code>frame_chksum_err</code>)
4	Filtru Kalman în execuție (<code>kalman_busy</code>)
5	Mod demo activ, reflectă <code>sw0</code>
6	Transmițător UART ocupat (<code>uart_tx_busy</code>)
7	SPI ocupat (<code>spi_busy_sig</code>)
8–15	Unghiul filtrat al servo 0 (8 biți)

LED-urile 8–15 afișează în binar unghiul filtrat al primului servo (degetul mic), oferind o verificare vizuală imediată a funcționării filtrului Kalman.

5.14.4. Logica de declanșare a răspunsului

Transmisia unui cadru de răspuns (readback) este declanșată atunci când un cadru valid cu direcția `0xFE` este recepționat:

```

1 process(clk)
2 begin
3     if rising_edge(clk) then
4         send_readback <= '0';
5         if frame_valid = '1' and frame_dir = x"FE" then
6             send_readback <= '1';
7         end if;
8     end if;
9 end process;
```

Listing 5.11: Logica de declanșare a cadrului de răspuns

Această logică sincronă generează un puls de un ciclu de ceas pe semnalul `send_readback`, care declanșează modulul `frame_tx` să transmită un cadru de răspuns conținând cele 8 unghiuri filtrate curente.

5.15. Proiectarea hardware — circuitele de interfață

Pe lângă implementarea FPGA, sistemul necesită circuite auxiliare de interfață pentru izolarea galvanică, alimentarea servomotoarelor și conversia de nivel logic.

5.15.1. Izolarea galvanică a semnalelor PWM — ADUM1400

Semnalele PWM generate de FPGA (la nivel logic de 3,3 V) trebuie convertite la 5 V pentru servomotoarele MG996R. Se utilizează două circuite integrate ADUM1400, fiecare oferind 4 canale de izolare galvanică digitală. ADUM1400 este un izolator bazat pe tehnologia *iCoupler* (transformatoare microfabricate), oferind:

- Izolare galvanică de 2 500 V_{rms}
- Timp de propagare tipic de 32 ns (neglijabil pentru semnale PWM)
- Conversie de nivel logic intrinsecă: alimentat la 3,3 V pe partea FPGA și la 5 V pe partea servomotoare

Primul ADUM1400 convertește semnalele PWM pentru servomotoarele 0–3 (JA0–JA3), iar al doilea pentru servomotoarele 4–7 (JB0–JB3).

5.15.2. Alimentarea servomotoarelor — DC2468A

Cele 8 servomotoare MG996R consumă fiecare până la 700 mA sub sarcină, totalizând un consum potențial de 5,6 A la 5 V. Sursa de alimentare utilizată este modulul DC2468A (bazat pe regulatorul LT3045), care convertește 12 V dintr-un alimentator extern la 5 V cu un curent maxim de 8 A. Această marjă de 30% asigură funcționarea stabilă chiar și când toate servomotoarele sunt acționate simultan.

5.15.3. Alimentarea circuitului ADC — ADP165

ADC-ul AD7124-8 și izolatorul SPI MAX14850 necesită alimentare la 3,3 V separată de domeniul FPGA. Se utilizează un regulator LDO ADP165 care furnizează 3,3 V cu zgomot ultra-scăzut din sursa de 5 V, minimizând impactul zgomotului de alimentare asupra preciziei măsurărilor ADC.

5.15.4. Izolarea SPI — MAX14850

Interfața SPI dintre FPGA și AD7124-8 traversează un izolator digital MAX14850 cu 4 canale. Acesta asigură izolarea galvanică între domeniul de ceas FPGA (3,3 V) și domeniul ADC (3,3 V alimentat separat), prevenind bucele de masă care ar introduce zgomot în măsurătorile ADC. Cele 4 canale sunt utilizate pentru: SCLK, MOSI, MISO și CS.

5.15.5. Conectarea servomotoarelor MG996R

Servomotoarele MG996R sunt conectate prin cabluri cu 3 fire:

- **Maro:** GND (masa comună)
- **Roșu:** V+ (5 V de la DC2468A)
- **Portocaliu:** Semnal PWM (5 V de la ADUM1400)

Fiecare MG996R dispune și de un al patrulea fir (alb), conectat la potențiometrul intern de feedback analogic. Acest semnal este direcționat către una din cele 8 intrări analogice ale AD7124-8 pentru citirea poziției reale a servomotorului.

Capitolul 6. Testare și validare

Acest capitol descrie metodologia de testare aplicată sistemului de control FPGA, acoperind atât testarea individuală a fiecărui modul cât și testarea integrată a lanțului complet de semnal. Se prezintă rezultatele măsurărilor, verificarea funcționalității și analiza utilizării resurselor FPGA.

6.1. Metodologia de testare

Strategia de testare urmează o abordare de tip *bottom-up*: fiecare modul VHDL este verificat individual înainte de a fi integrat în sistemul complet. Această metodologie permite izolarea defectelor la nivelul modului unde au origine, simplificând depanarea.

Testarea s-a desfășurat pe două niveluri:

1. **Testare per modul:** Fiecare modul este instanțiat izolat în modulul de nivel superior, cu semnale de intrare controlate (hardcodate sau furnizate de un generator de stimuli) și ieșiri observate pe LED-uri, osciloscop sau terminal serial.
2. **Testare de integrare:** Modulele sunt conectate conform arhitecturii finale, iar sistemul complet este exercitat prin scenarii end-to-end: cadre UART transmise de la PC-ul gazdă, verificarea unghiurilor pe osciloscop la ieșirile PWM și monitorizarea indicatoarelor LED.

Instrumentele utilizate:

- Osciloscop digital pentru măsurarea semnalelor PWM și SPI
- Terminal serial (PuTTY / Tera Term) la 115 200 baud pentru comunicarea UART
- LED-urile plăcii Basys3 pentru monitorizarea stărilor interne
- Vivado Hardware Manager pentru programarea și reprogramarea rapidă a bitstreamului

6.2. Testarea modului PWM

Modulul `pwm_gen` a fost primul testat deoarece generarea corectă a semnalelor PWM este condiția fundamentală pentru controlul servomotoarelor.

6.2.1. Metodă

S-a instanțiat o singură instanță a modului `pwm_gen` cu unghiul hardcodat la diferite valori de test. Ieșirea PWM a fost conectată la un pin Pmod și măsurată cu osciloscopul.

6.2.2. Cazuri de test

Tabela 6.1: Rezultatele testării modulului PWM

Unghi intrare	Lățime puls așteptată	Lățime puls măsurată	Eroare
0°	1,000 ms	1,000 ms	<10 ns
90°	1,500 ms	1,500 ms	<10 ns
180°	2,001 ms	2,001 ms	<10 ns
45°	1,250 ms	1,249 ms	<10 μ s
135°	1,750 ms	1,751 ms	<10 μ s

6.2.3. Analiza rezultatelor

Verificarea la 90° este deosebit de relevantă deoarece corespunde poziției mediane a servomotorului:

$$\text{threshold}_{90} = 100\,000 + 90 \times 556 = 100\,000 + 50\,040 = 150\,040 \quad (6.1)$$

La 100 MHz, 150 040 de cicluri corespund unei lățimi de puls de $150\,040 \times 10\text{ ns} = 1\,500\,400\text{ ns} \approx 1,500\text{ ms}$, ceea ce este corect.

Eroarea minimă observată (sub un ciclu de ceas de 10 ns) confirmă precizia generatorului PWM hardware. Comparativ, o implementare software pe microcontroler ar prezenta jitter de ordinul microsecundelor din cauza întreruperilor și a schimbărilor de context.

Perioada semnalului PWM a fost măsurată la exact 20,000 ms (± 10 ns), confirmând funcționarea corectă a numărătorului pe 21 de biți.

6.3. Testarea comunicației UART

6.3.1. Testul de loopback

Pentru verificarea izolată a modulelor `uart_rx` și `uart_tx`, s-a implementat o configurație de loopback: fiecare octet recepționat este retransmis imediat prin `uart_tx`. Conectarea se realizează prin cablul USB integrat pe placa Basys3, care include un convertor USB-la-UART.

6.3.2. Procedura de test

1. Se deschide PuTTY (sau Tera Term) cu parametrii: portul COM corespunzător plăcii Basys3, 115 200 baud, 8N1 (8 biți de date, fără paritate, 1 bit de stop).
2. Se activează opțiunea *local echo* în terminalul serial pentru a vizualiza caracterele transmise alături de cele recepționate.
3. Se tastează caractere individuale și secvențe de text. Fiecare caracter transmis trebuie să apară de două ori pe ecran: o dată de la echo-ul local și o dată de la loopback-ul FPGA.
4. Se transmit secvențe rapide de caractere pentru a verifica funcționarea la debit maxim.

6.3.3. Rezultate

Toate caracterele au fost recepționate și retransmise corect, inclusiv la debit maxim continuu. Nu s-au observat pierderi de caractere sau erori de recepție. Testul confirmă:

- Funcționarea corectă a sincronizatorului cu dublu flip-flop (nicio eroare de metastabilitate detectată)
- Eșantionarea la mijlocul bitului funcționează corespunzător
- Calculul parametrilor de baud ($\text{CLKS_PER_BIT} = 867$) este corect
- Transmisia și recepția LSB-first sunt implementate conform standardului

6.4. Testarea protocolului de cadre

6.4.1. Verificarea recepției cadrelor

Testarea modulului `frame_rx` s-a realizat prin transmiterea de cadre de 10 octeți de la PC prin intermediul unui script Python care construiește cadre cu unghiuri cunoscute și calculează checksum-ul XOR corect.

Cazul 1: Cadru valid

S-a transmis un cadru cu toți servo-urile la 90° (0x5A):

FF 5A 5A 5A 5A 5A 5A 5A 5A [checksum]

Checksum = $\text{FF} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} \oplus 5\text{A} = \text{A5}$.

Cadrul complet: FF 5A 5A 5A 5A 5A 5A 5A 5A A5.

Rezultat observat:

- LED 2 (`frame_valid`) pulsează scurt la recepția cadrului
- LED 3 (`checksum_err`) rămâne stins
- LED-urile 8–15 afișează valoarea 0x5A (90) în binar
- Servomotorul conectat se poziționează la 90°

Cazul 2: Cadru cu checksum eronat

S-a transmis același cadru, dar cu octetul de checksum modificat intenționat (un bit inversat):

FF 5A 5A 5A 5A 5A 5A 5A A4 (checksum corect ar fi A5)

Rezultat observat:

- LED 2 (`frame_valid`) rămâne stins
- LED 3 (`checksum_err`) pulsează scurt
- Pozițiile servomotoarelor nu se modifică (mailbox-ul nu primește date noi)

Cazul 3: Cadru incomplet (test timeout)

S-au transmis doar primii 5 octeți ai unui cadru, apoi s-a așteptat mai mult de 1 ms fără a continua. Rezultat: modulul revine automat la starea `S_WAIT_DIR`, iar la transmiterea unui cadru complet ulterior, acesta este recepționat corect. Mecanismul de timeout previne blocarea permanentă a parser-ului.

6.4.2. Verificarea transmisiei cadrelor de răspuns

S-a transmis un cadru cu direcția 0xFE (cerere de citire). FPGA-ul a răspuns cu un cadru de 10 octeți conținând 0xFE + cele 8 unghiuri filtrate curente + checksum XOR. Cadrul recepționat pe PC a fost verificat: checksum-ul a fost corect, iar unghiurile au corespuns cu valorile afișate pe LED-uri.

6.5. Testarea SPI și ADC

6.5.1. Verificarea interfeței SPI

Testarea modului `spi_master` s-a realizat prin monitorizarea semnalelor SPI pe osciloscop, cu sondele conectate la pinii Pmod JC (SCLK, MOSI, MISO, CS).

S-au verificat:

- **Polaritatea SCLK:** SCLK este inactiv la nivel '1' (CPOL=1), confirmat vizual pe osciloscop
- **Fazarea datelor:** MOSI se modifică pe frontul descrescător al SCLK, iar MISO este eșantionat pe frontul crescător (CPHA=1)
- **Frecvența SCLK:** Măsurată la 5 MHz ($\pm 0,1$ MHz), conform calcului $100 \text{ MHz} / 20$
- **Ordinea biților:** MSB first, verificat prin compararea secvenței de biți pe osciloscop cu valoarea transmisă

6.5.2. Verificarea comunicării cu AD7124-8

Primul test de comunicare a constatat în citirea registrului de identificare (ID register, adresa 0x05) al AD7124-8. Valoarea citită a fost comparată cu valoarea specificată în fișa de catalog (0x14 pentru AD7124-8). Citirea corectă a ID-ului confirmă:

- Funcționarea corectă a masterului SPI
- Integritatea izolării SPI prin MAX14850
- Conexiunea electrică corectă între FPGA și ADC

6.5.3. Verificarea scanării canalelor

După inițializarea completă a ADC-ului, s-a verificat că toate cele 8 canale sunt scanate ciclic prin monitorizarea biților 3:0 ai octetului de status. Secvența observată (0, 1, 2, 3, 4, 5, 6, 7, 0, 1, ...) confirmă configurarea corectă a tuturor registrelor de canal.

LED-ul 0 (`adc_cfg_done`) se aprinde la finalizarea secvenței de inițializare, iar LED-ul 1 (`meas_valid`) pulsează periodic la fiecare ciclu complet de citire a tuturor celor 8 canale.

6.6. Testarea filtrului Kalman

Filtrul Kalman a fost testat prin trei categorii de teste: convergență, răspuns la treapta și rejectia zgomotului.

6.6.1. Testul de convergență

Obiectiv: Verificarea că filtrul converge de la starea inițială (incertitudine mare, $P_{\text{init}} = 10, 0$) la o estimare stabilă.

Procedură: S-a furnizat o măsurătoare constantă de 90° pe toate cele 8 canale. S-a monitorizat evoluția unghiului filtrat pe LED-urile 8–15 (care afișează unghiul servo 0).

Rezultat: După aproximativ 20 de cicluri ($20 \times 20 \text{ ms} = 400 \text{ ms}$), unghiul filtrat s-a stabilizat la 90° . În primele cicluri, valoarea a crescut rapid de la 0° (valoarea inițială) către 90° , cu o tranziție exponențială caracteristică filtrului Kalman. Comportamentul este conform cu teoria: incertitudinea inițială mare ($P_{00} = 10, 0$) face ca câștigul Kalman K_0 să fie aproape 1,0 în primele cicluri, permițând filtrului să „creadă” rapid măsurătoarea. Pe măsură ce P_{00} scade, câștigul se stabilizează la o valoare intermediară.

6.6.2. Testul de răspuns la treaptă

Obiectiv: Verificarea comportamentului filtrului la o schimbare bruscă a unghiului (de la 0° la 180°).

Procedură: După stabilizarea filtrului la 0° , s-a transmis un cadru UART cu unghiul 180° . S-a monitorizat evoluția ieșirii filtrate.

Rezultat: Filtrul a atins 90% din valoarea finală (162°) în aproximativ 10 cicluri (200 ms) și s-a stabilizat complet la 180° în aproximativ 15 cicluri (300 ms). Comportamentul este un compromis între răspunsul rapid (determinat de R mic, adică încredere în măsurătoare) și stabilitatea (determinată de Q mic, adică încredere în predicție). Componentele de viteză (x_1) au demonstrat o creștere tranzitorie în timpul tranziției, confirmând că mecanismul de învățare a vitezei funcționează corect (ecuația 5.14).

6.6.3. Testul de reacție a zgomotului

Obiectiv: Compararea ieșirii filtrate cu măsurătoarea brută ADC pentru a evalua capacitatea de filtrare.

Procedură: S-a poziționat manual un servomotor la un unghi fix și s-au citit simultan valoarea brută ADC (prin cadre de răspuns 0xFE) și valoarea filtrată (de pe LED-uri și din unghiurile PWM).

Rezultat: Valoarea brută ADC a prezentat fluctuații de $\pm 2\text{--}3^\circ$ datorită zgomotului analogic și cuantizării. Ieșirea filtrată a prezentat fluctuații de maximum $\pm 0,5^\circ$, demonstrând o reducere a zgomotului cu factor de 4–6 $\times$. Servomotorul controlat de ieșirea filtrată a fost vizibil mai stabil decât un servomotor controlat direct de valoarea brută.

6.7. Testarea modului demo

6.7.1. Procedură

1. Se comută sw0 în poziția ON pe placa Basys3.
2. Se verifică LED 5 (demo mode active) — trebuie aprins.
3. Se observă secvențele de mișcare ale servomotoarelor.
4. Se comută sw0 înapoi în poziția OFF pentru a verifica dezactivarea.

6.7.2. Rezultate

Toate secvențele demonstrative au fost executate corect:

- **Deschidere/închidere pumn:** Toate cele 4 degete (pinky, ring, middle, index) se deschid și se închid simultan, cu mișcarea durând 1 secundă per tranziție.
- **Fluturare:** Încheietura rotește alternativ stânga și dreapta, revenind la centru, cu o durată de 0,5 secunde per direcție.
- **Flexii individuale:** Fiecare deget se închide individual în ordine (pinky, ring, middle, index), cu celelalte rămânând deschise.
- **Thumbs up:** Degetele se închid, iar degetul mare rămâne extins — gest vizual

corect.

- **Pointing:** Degetul arătător rămâne extins, restul se închid.
- **Flexia cotului:** Servomotorul de cot flexează și extinde.

La dezactivarea sw0, secvența se oprește imediat, iar la reactivare pornește de la începutul ciclului (cadru-cheie 0). Tranziția între modul demo și modul UART se realizează fără discontinuități datorită filtrării Kalman care netezește orice schimbare bruscă de unghi.

6.8. Testare de integrare

6.8.1. Lanțul complet de semnal

Testarea de integrare a verificat funcționarea lanțului complet:

Cadru UART → frame_rx → mailbox → source_mux → kalman_engine → pwm_gen → ADUM1400 → Servomotor MG996R

Procedură: S-au transmis cadre UART cu unghiuri variabile de la PC, generând mișcări ale servomotoarelor. S-a verificat vizual că mișcarea mâinii robotice corespunde unghiurilor comandate.

6.8.2. Măsurarea latenței end-to-end

Latența totală de la recepția unui cadru UART până la modificarea efectivă a semnalului PWM a fost estimată:

Tabela 6.2: Contribuțiile la latența end-to-end

Etapă	Durată
Recepție UART (10 octeți × 86,8 μs)	868 μs
Parsare cadru (instantanee, pipeline)	<1 μs
Depunere mailbox (instantanee)	<1 μs
Așteptare tick 20 ms (caz mediu)	10 ms
Execuție filtru Kalman (8 canale)	≈14,6 μs
Propagare la ieșire PWM	<1 μs
Total (caz mediu)	≈10,9 ms
Total (caz cel mai defavorabil)	≈20,9 ms

Latența dominantă este așteptarea tick-ului de 20 ms în mailbox. Aceasta este inherentă arhitecturii servo (ciclul de 50 Hz) și nu poate fi redusă fără a modifica perioadă de bază. Latența de procesare propriu-zisă a FPGA-ului (parsare + Kalman + PWM) este sub 15 μs, cu trei ordine de mărime mai mică decât ciclul servo.

6.8.3. Verificarea indicatoarelor LED

S-au verificat toate combinațiile de stări LED în scenariile de funcționare:

Tabela 6.3: Starea LED-urilor în diferite scenarii de funcționare

Scenariu	L0	L1	L2	L3	L4	L5	L6	L7
După pornire	ON	blink	—	—	blink	OFF	—	blink
Cadru valid	ON	blink	blink	—	blink	OFF	—	blink
Eroare checksum	ON	blink	—	blink	blink	OFF	—	blink
Mod demo	ON	blink	—	—	blink	ON	—	blink
Răspuns 0xFE	ON	blink	blink	—	blink	OFF	blink	blink

Notă: „blink” indică pulsuri scurte periodice, „—” indică absența activității pe acel LED. LED-ul 0 (ADC config done) rămâne permanent aprins după inițializare, confirmând configurarea reușită.

6.9. Utilizarea resurselor FPGA

Tabelul 6.4 prezintă utilizarea resurselor FPGA după sinteză și implementare pentru circuitul integrat Xilinx Artix-7 XC7A35T (prezent pe placa Basys3).

Tabela 6.4: Utilizarea resurselor FPGA (Artix-7 XC7A35T)

Resursă	Disponibil	Utilizat	Procent
LUT-uri (Look-Up Tables)	20 800	~3 000	14%
Flip-flop-uri	41 600	~2 500	6%
Block RAM (BRAM36)	50	2	4%
DSP48E1	90	2–4	4%

6.9.1. Analiza utilizării

Gradul de utilizare global al FPGA-ului este redus (sub 15% pe orice categorie de resursă), ceea ce oferă mai multe avantaje:

1. **Marjă pentru extinderi:** Resursele rămase permit adăugarea de module suplimentare fără a necesita migrarea la un FPGA mai mare. De exemplu, un filtru Kalman cu 3 stări (adăugarea accelerației) ar necesita doar 2–4 DSP48E1 suplimentare.
2. **Performanță temporală:** Gradul scăzut de utilizare a LUT-urilor facilitează rutarea cu căi de propagare scurte, garantând respectarea constrângerilor de timing la 100 MHz fără dificultăți.
3. **Consum energetic redus:** Resursele neutilizate nu contribuie la consumul dinamic, menținând disiparea termică la un nivel scăzut.

6.9.2. Distribuția resurselor per modul

Principalii consumatori de resurse sunt:

- **kalman_engine:** Domină utilizarea LUT-urilor datorită aritmeticii Q16.16 (multiplicări, adunări pe 32 de biți) și a memoriei de stare (40 cuvinte $\times$ 32 biți). Blocurile DSP48E1 sunt utilizate pentru multiplicările implementate de funcția `mul_q16` și pentru operațiile de diviziune.
- **ad7124_ctrl:** Consumă o parte semnificativă din flip-flop-uri datorită numărului mare de registre de stare (secvența de inițializare, numărătoare, registre de date).

- **pwm_gen (8 instanțe):** Fiecare instanță necesită un numărător pe 21 de biți și un comparator, totalizând $8 \times 21 = 168$ flip-flop-uri și logica de comparație aferentă.
- **demo_rom:** ROM-ul cu 21 de cadre-cheie este sintetizat în LUT-uri (nu în Block RAM) deoarece dimensiunea sa mică nu justifică utilizarea unui bloc BRAM dedicat.

Cele 2 blocuri Block RAM sunt utilizate de sintetizator pentru implementarea memoriei de stare a filtrului Kalman (`state_ram`, 40 cuvinte $\times$ 32 biți) și pentru registrele interne ale controlerului ADC.

Capitolul 7. Manual de instalare și utilizare

Acest capitol oferă instrucțiunile necesare pentru reproducerea, configurarea și utilizarea sistemului de control FPGA pentru mâna robotică. Se prezintă cerințele hardware și software, procedura de instalare a mediului de dezvoltare, programarea plăcii FPGA, conectarea componentelor hardware și modurile de operare ale sistemului.

7.1. Cerințe hardware

Componentele hardware necesare pentru construirea și operarea sistemului sunt enumerate în Tabelul 7.1.

Tabela 7.1: Lista componentelor hardware necesare

Componentă	Descriere	Cantitate
Basys3	Placă FPGA Xilinx Artix-7 XC7A35T, 100 MHz	1
Cablu USB	Micro-USB pentru programare și UART	1
MG996R	Servomotor cu cuplu mare, feedback analogic	8
ADUM1400	Izolator galvanic digital, 4 canale	2
DC2468A	Convertor step-down 12 V $\rightarrow$ 5 V, 8 A	1
ADP165	Regulator LDO 5 V $\rightarrow$ 3,3 V, zgomot scăzut	1
AD7124-8	ADC sigma-delta pe 24 biți, 8 canale	1
MAX14850	Izolator SPI digital, 4 canale	1
Alimentator	12 V DC, minim 8 A	1
Fire de conexiune	Dupont, pentru interconectarea modulelor	diverse

Observație importantă: Alimentatorul de 12 V trebuie să fie capabil să furnizeze cel puțin 8 A, deoarece cele 8 servomotoare MG996R pot consuma simultan până la 5,6 A sub sarcină maximă. Placa Basys3 este alimentată separat prin cablu USB și nu trebuie conectată la sursa de 12 V.

7.2. Cerințe software

- **Xilinx Vivado 2025.2** (sau versiune ulterioară) — Mediul de dezvoltare pentru sinteză, implementare și programarea FPGA. Ediția *WebPack* (gratuită) este suficientă, deoarece circuitul XC7A35T este inclus în licența WebPack.
- **Drivere Digilent** — Necesare pentru ca Vivado să recunoască placa Basys3 prin Hardware Manager. Acestea sunt incluse în instalarea Vivado, dar pot necesita instalare separată în anumite configurații.

- **Terminal serial** — PuTTY, Tera Term sau orice emulator de terminal serial capabil de 115 200 baud, 8N1. Utilizat pentru testarea comunicării UART și transmiterea cadrelor de comandă.

7.3. Instalarea mediului de dezvoltare

7.3.1. Crearea proiectului RTL

1. Se lansează Vivado și se selectează **Create Project**.
2. Se alege tipul proiectului: **RTL Project**.
3. La pasul de selecție a dispozitivului, se caută și se selectează: **xc7a35tcpg236-1** (Artix-7, pachetul CPG236, speed grade –1).
4. Se adaugă toate cele 13 fișiere sursă VHDL: `system_tick_gen.vhd`, `uart_rx.vhd`, `uart_tx.vhd`, `frame_rx.vhd`, `frame_tx.vhd`, `mailbox.vhd`, `source_mux.vhd`, `demo_rom.vhd`, `spi_master.vhd`, `ad7124_ctrl.vhd`, `kalman_engine.vhd`, `pwm_gen.vhd`, `basys3_top.vhd`.
5. Se adaugă fișierul de constrângeri: `basys3_servo.xdc`.
6. Se setează `basys3_top` ca modul de nivel superior (*top module*) prin clic dreapta → **Set as Top**.

7.3.2. Rezolvarea problemelor comune

Eroare de tip spawn la sinteză

În mediul WSL2 sau în configurații cu resurse limitate, Vivado poate genera erori de tip *spawn* la lansarea proceselor paralele de sinteză. Soluția constă în executarea sintezei și implementării prin consola TCL cu un singur fir de execuție:

```

1 reset_run synth_1
2 launch_runs synth_1 -jobs 1
3 wait_on_run synth_1
4 launch_runs impl_1 -jobs 1
5 wait_on_run impl_1
6 launch_runs impl_1 -to_step write_bitstream -jobs 1
7 wait_on_run impl_1

```

Listing 7.1: Sinteza și implementare prin consola TCL cu un singur job

Eroare hw_server blocat

Dacă Hardware Manager nu se poate conecta la placa Basys3, cauza uzuală este un proces `hw_server` rezidual din o sesiune anterioară. Soluția:

1. Se închid toate instanțele Vivado.
2. Se opresc procesele `hw_server.exe` și `cs_server.exe` din Task Manager (Windows) sau prin comanda `kill` (Linux).
3. Se relansează Vivado și se reconectează Hardware Manager.

Placa Basys3 nedetectată

Dacă placa nu este detectată la conectarea USB, se verifică instalarea driverelor Digilent. Calea implicită a instalatorului este:

C:\Xilinx\Vivado\2025.2\data\xicom\cable_drivers\nt64\install_digilent.exe

Se execută acest program cu drepturi de administrator.

7.4. Programarea plăcii

Procesul de generare și încărcare a bitstreamului se realizează în următorii pași:

1. **Sinteză:** Se selectează **Run Synthesis** din panoul *Flow Navigator* (sau tasta F11). Procesul de sinteză traduce codul VHDL în primitive logice (LUT-uri, flip-flop-uri, blocuri DSP). Durata tipică: 1–3 minute.
2. **Implementare:** Se selectează **Run Implementation**. Implementarea plasează primitivele logice pe resursele fizice ale FPGA-ului și rutează interconexiunile. Se verifică raportul de timing: toate căile trebuie să respecte constrângerea de 10 ns (100 MHz). Durata tipică: 2–5 minute.
3. **Generare bitstream:** Se selectează **Generate Bitstream**. Acesta generează fișierul `.bit` care codifică întreaga configurare a FPGA-ului.
4. **Programare dispozitiv:** Se deschide **Hardware Manager** → **Open Target** → **Auto Connect**. Se selectează dispozitivul detectat (xc7a35t) și se alege **Program Device**. Se specifică fișierul `.bit` generat la pasul anterior.

Notă: Configurarea FPGA-ului este volatilă — la deconectarea alimentării, bitstreamul se pierde și trebuie reîncărcat. Placa Basys3 dispune de memorie Flash QSPI pentru stocarea permanentă a bitstreamului, dar aceasta necesită pași suplimentari de configurare care depășesc scopul acestui manual.

7.5. Conectarea hardware

7.5.1. Conectarea servomotoarelor prin izolatori

Conectorul JA (servouri 0–3):

- JA0 → ADUM1400 #1 intrare canal 1 → ieșire canal 1 → servo 0 (pinky) — fir portocaliu
- JA1 → ADUM1400 #1 canal 2 → servo 1 (ring)
- JA2 → ADUM1400 #1 canal 3 → servo 2 (middle)
- JA3 → ADUM1400 #1 canal 4 → servo 3 (index)

Conectorul JB (servouri 4–7):

- JB0 → ADUM1400 #2 canal 1 → servo 4 (thumb-palm)
- JB1 → ADUM1400 #2 canal 2 → servo 5 (thumb-tension)
- JB2 → ADUM1400 #2 canal 3 → servo 6 (wrist)
- JB3 → ADUM1400 #2 canal 4 → servo 7 (elbow)

Partea primară a fiecărui ADUM1400 se alimentează la 3,3 V (de la pinii VCC ai conectorilor Pmod), iar partea secundară la 5 V (de la DC2468A).

Conectorul JC (SPI la ADC):

- JC0 → MAX14850 → AD7124-8 CS
- JC1 → MAX14850 → AD7124-8 DIN (MOSI)
- JC2 ← MAX14850 ← AD7124-8 DOUT (MISO)
- JC3 → MAX14850 → AD7124-8 SCLK

7.5.2. Alimentarea

- **Basys3:** Alimentat prin cablu USB (5 V), logic la 3,3 V.

- **Servomotoare:** Alimentate la 5 V de la DC2468A, care primește 12 V de la alimentatorul extern. Masa (GND) este comună cu Basys3 prin conectorii Pmod.
- **ADC și izolator SPI:** Alimentate la 3,3 V de la ADP165, care primește 5 V de la DC2468A. Această alimentare separată minimizează zgomotul indus de servomotoare asupra măsurărilor ADC.

Atenție: Se conectează mai întâi masa (GND) între toate modulele înainte de a conecta tensiunile de alimentare, pentru a preveni diferențele de potențial pe liniile de date.

7.6. Utilizarea sistemului

7.6.1. Modul demonstrativ

Modul demonstrativ permite testarea rapidă a sistemului fără a necesita conexiunea la un calculator gazdă.

1. Se programează placa Basys3 cu bitstreamul generat.
2. Se comută **sw0** (cel mai din dreapta comutator) în poziția ON.
3. LED-ul 5 se aprinde, confirmând activarea modului demo.
4. Servomotoarele execută secvențial cele 21 de cadre-cheie: deschidere/închidere pumn, fluturare, flexii individuale, thumbs up, pointing, flexia cotului.
5. Secvența se repetă continuu până la dezactivarea **sw0**.

7.6.2. Modul UART (control de la PC)

1. Se asigură că **sw0** este în poziția OFF.
2. Se conectează placa Basys3 la PC prin cablu USB (același cablu care asigură și alimentarea).
3. Se deschide un terminal serial cu parametrii:
 - Port: portul COM corespunzător plăcii Basys3
 - Baud rate: 115 200
 - Format: 8N1 (8 biți date, fără paritate, 1 bit stop)
4. Se transmite cadre de 10 octeți conform protocolului definit în Capitolul 4:
 - Octet 0: 0xFF (scriere unghiuri) sau 0xFE (citire)
 - Octeții 1–8: unghiuri 0–180 pentru cele 8 servouri
 - Octet 9: checksum XOR al octeților 0–8
5. LED 2 pulsează la fiecare cadru valid recepționat.

7.6.3. Resetarea sistemului

Apăsarea butonului **btnC** (butonul central) de pe placa Basys3 efectuează un reset sincron al întregului sistem. Acest lucru:

- Resetează filtrul Kalman (revenire la starea inițială cu incertitudine mare)
- Resetează ADC-ul (reinițializare completă a registrelor)
- Resetează modulul UART (gata pentru recepție)
- Resetează modulul demo (pornire de la cadrul-cheie 0)
- Setează toate ieșirile PWM la 0° (puls de 1 ms)

LED-ul 0 se stinge momentan în timpul reinițializării ADC-ului (aproximativ 5 ms), apoi se reaprinde când configurarea este finalizată.

7.6.4. Interpretarea indicatoarelor LED

- **LED 0 aprins permanent:** ADC inițializat corect — funcționare normală.
- **LED 0 stins:** ADC nu s-a inițializat — verificați conexiunea SPI și alimentarea.
- **LED 2 pulsează:** Cadre UART recepționate cu succes.
- **LED 3 pulsează:** Erori de checksum — verificați integritatea cablului USB sau corectitudinea calculului de checksum.
- **LED 5 aprins:** Mod demo activ (sw0 = ON).
- **LED-urile 8–15:** Afișează în binar unghiul filtrat al servo 0 (ex: 01011010 = 90°).

Capitolul 8. Concluzii

8.1. Contribuții

Prezenta lucrare a realizat proiectarea și implementarea completă a unui sistem de control FPGA pentru o mână robotică cu 8 servomotoare, integrând filtrare Kalman pentru stabilizarea mișcării. Principalele contribuții sunt:

1. **Control în timp real pe FPGA cu filtrare Kalman.** S-a implementat un filtru Kalman cu 2 stări (poziție și viteză) în aritmetică virgula fixă Q16.16, multiplexat în timp pe 8 canale. Filtrul estimează optimal traiectoria fiecărui servomotor pe baza măsurărilor zgomotoase, reducând fluctuațiile cu un factor de $4\text{--}6\times$ comparativ cu controlul direct. Întregul ciclu de filtrare pentru cele 8 canale se execută în aproximativ 1456 cicluri de ceas ($14,6\mu\text{s}$), reprezentând doar 0,07% din bugetul temporal disponibil de 20 ms.
2. **Generare PWM paralelă pe 8 canale.** Cele 8 semnale PWM sunt generate simultan de instanțe hardware independente, fără multiplexare în timp. Aceasta elimină jitter-ul de interleaving specific implementărilor bazate pe microcontroler, rezultând o precizie temporală de 10 ns (un ciclu de ceas) pe fiecare canal.
3. **Integrarea feedback-ului ADC.** Controlerul AD7124-8 citește poziția reală a celor 8 servomotoare prin potențiometrele de feedback, furnizând măsurători pe 24 de biți la o rată de 50 Hz. Aceste măsurători alimentează filtrul Kalman, permițând estimarea nu doar a poziției, ci și a vitezei de deplasare.
4. **Mod demonstrativ autonom.** ROM-ul cu 21 de cadre-cheie permite funcționarea autonomă a sistemului, fără conexiune la un calculator gazdă, facilitând demonstrarea și testarea rapidă.
5. **Arhitectură modulară completă.** Cele 12 module VHDL sunt proiectate cu interfețe clar definite și pot fi reutilizate independent în alte proiecte. Gradul scăzut de utilizare a resurselor FPGA (sub 15%) lasă marjă substanțială pentru extinderi ulterioare.

8.2. Analiză critică

8.2.1. Puncte forte

- **Latență redusă:** Latența de procesare a FPGA-ului (de la recepția cadrului UART la actualizarea semnalelor PWM) este sub $15\mu\text{s}$, cu trei ordine de mărime mai mică decât ciclul servo de 20 ms. Aceasta garantează că procesarea nu introduce întârzieri perceptibile.
- **Paralelism real:** Generarea simultană a tuturor canalelor PWM, funcționarea concurrentă a receptorului UART, a controlerului SPI și a filtrului Kalman — toate în același ciclu de ceas — demonstrează avantajul fundamental al FPGA față de un procesor secvențial.
- **Utilizare eficientă a resurselor:** Sistemul complet ocupă sub 15% din resursele FPGA-ului Artix-7 XC7A35T, cel mai mic circuit din familia Artix-7. Aceasta demonstrează că implementarea este eficientă din punct de vedere al resurselor

hardware.

- **Robustețe la comunicare:** Protocolul de cadre cu checksum XOR, timeout de 1 ms și buffer dublu (mailbox) asigură funcționarea stabilă chiar și în condiții de comunicare UART imperfectă.

8.2.2. Limitări

- **Filtru Kalman cu 2 stări:** Modelul curent nu include accelerația ca variabilă de stare. Pentru mișcări rapide cu accelerație variabilă, estimarea poate rămâne în urma mișcării reale. Un filtru cu 3 stări (poziție, viteză, accelerație) ar captura mai fidel dinamica mișcărilor brusieri, dar necesită o matrice de covarianță 3×3 (6 elemente unice) în loc de 2×2 (3 elemente).
- **Constante de ajustare fixe:** Constantele filtrului Kalman (Q_{00} , Q_{11} , R) sunt hardcodeate în VHDL și necesită resinteză pentru modificare. Valorile actuale sunt calculate teoretic și ar beneficia de calibrare pe hardware-ul real, eventual cu un mecanism de ajustare în timp real prin UART.
- **Comunicare doar prin cablu:** Sistemul depinde de o conexiune USB cablată pentru comunicarea UART cu calculatorul gazdă. Aceasta limitează mobilitatea operatorului și introduce o dependență fizică de proximitatea calculatorului.
- **Configurare ADC fără validare:** Secvența de inițializare a AD7124-8 scrie registrele de configurare fără a citi înapoi valorile pentru confirmare. O eroare de comunicare SPI în timpul inițializării ar putea lăsa ADC-ul într-o stare incorectă fără detecție.

8.3. Dezvoltări ulterioare

Arhitectura modulară și gradul scăzut de utilizare a resurselor FPGA permit mai multe direcții de extindere:

1. **Filtru Kalman cu 3 stări.** Adăugarea accelerației ca a treia variabilă de stare ar îmbunătăți urmărirea mișcărilor rapide. Vectorul de stare devine $[x_0, x_1, x_2]$ (poziție, viteză, accelerație), iar matricea de covarianță crește de la 2×2 la 3×3 . Termenul $dt^2/2$ (0,0002 în Q16.16) necesită o reprezentare cu mai mulți biți fracționari (Q4.28) pentru a evita pierderea de precizie. Estimarea de resurse indică 45 multiplicări și 3 diviziuni per canal, încadrabile confortabil în bugetul temporal existent.
2. **Comunicare wireless.** Înlocuirea legăturii USB-UART cu un modul Bluetooth (ex. HC-05) sau WiFi (ex. ESP32) ar permite operarea la distanță. Modulele `uart_rx` și `uart_tx` ar rămâne nemodificate, deoarece aceste module wireless comunică prin UART standard. Singura modificare ar fi la nivel de placă, conectând modulul wireless la pinii RsRx/RsTx în locul convertorului USB-UART integrat.
3. **Bucă de control PID.** Combinarea ieșirii filtrului Kalman (estimarea poziției reale) cu unghiul țintă (din cadrul UART) într-un controler PID ar permite compensarea activă a erorilor de poziționare ale servomotoarelor. Eroarea $e = \theta_{\text{țintă}} - \theta_{\text{Kalman}}$ ar alimenta un controler PID a cărui ieșire ar ajusta comanda PWM, închizând bucla de reglare.
4. **Clasificare gesturi prin învățare automată.** Integrarea unui clasificator simplu (rețea neuronală compactă sau arbore de decizie) implementat în FPGA ar permite recunoașterea gesturilor direct pe placă, fără a depinde de procesarea MediaPipe pe un calculator gazdă. Gesturile recunoscute ar putea declanșa secvențe predefinite, similar modulului demo actual, dar bazate pe feedback-ul ADC în timp real.
5. **Extinderea numărului de grade de libertate.** Arhitectura actuală suportă 8 ca-

nale, dar poate fi extinsă la 16 sau mai multe prin adăugarea de instanțe PWM și extinderea filtrului Kalman. Resursele FPGA disponibile (peste 85% neutilizate) permit dublarea numărului de canale fără dificultăți.

Bibliografie

- [1] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, “MediaPipe: A framework for building perception pipelines,” in *Workshop on Computer Vision for AR/VR at IEEE CVPR*, 2019.
- [2] Horizon, “I built a bionic hand from scratch,” Material video, <https://www.youtube.com/watch?v=aHFo-7ZK1Bk>, 2024, accesat: 2026.
- [3] M. E. Hussein, “3D printed myoelectric prosthetic arm,” Master’s thesis, The University of Sydney, School of Aerospace, Mechanical and Mechatronic Engineering, 2014.
- [4] Egypt Museum, “Prosthetic toe of tabaketenmut,” <https://egypt-museum.com/prosthetic-toe-of-tabaket-en-mut/>, accesat: 2026.
- [5] MedlinePlus Magazine, “Prosthetics through the ages,” <https://magazine.medlineplus.gov/article/prosthetics-through-the-ages>, accesat: 2026.
- [6] Lakshita, “How new technology advancements have enhanced the modern prosthetic devices,” <https://circuitdigest.com/article/how-new-technology-advancements-have-enhanced-the-modern-prosthetic-devices>, accesat: 2026.
- [7] V. AUTORI], “Design of a highly biomimetic anthropomorphic robotic hand towards artificial limb regeneration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2016, iEEE Xplore, art. no. 7487528.
- [8] Dr. Gordon Groh, “Extensor tendon injuries,” <https://www.drgordongroh.com/orthopaedic-injuries-treatment/hand-wrist/extensor-tendon-injuries/>, accesat: 2026.
- [9] G. Langevin, “InMoov – open source 3D printed life-size robot,” <https://inmoov.fr/>, 2014.
- [10] Ryonicle, “Model de mână robotică imprimabilă 3D,” <https://www.thingiverse.com/thing:7111274>, accesat: 2026.
- [11] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.
- [12] G. Welch and G. Bishop, “An introduction to the Kalman filter,” University of North Carolina at Chapel Hill, Tech. Rep. TR 95-041, 1995.
- [13] A. Zucconi, “The mathematics of the Kalman filter,” <https://www.alanzucconi.com/2022/07/24/kalman-gain/>, 2022.

- [14] Tower Pro, *MG996R High Torque Metal Gear Dual Ball Bearing Servo*, Tower Pro Pte Ltd., 2018.
- [15] L. A. Jones and S. J. Lederman, *Human Hand Function*. Oxford University Press, 2006.

Anexa A. Codul sursă VHDL

Codul sursă complet al tuturor modulelor VHDL este disponibil în depozitul public al proiectului: <https://github.com/CioRadu1/FPGA-KalmanFilter-For-HandMovements>.